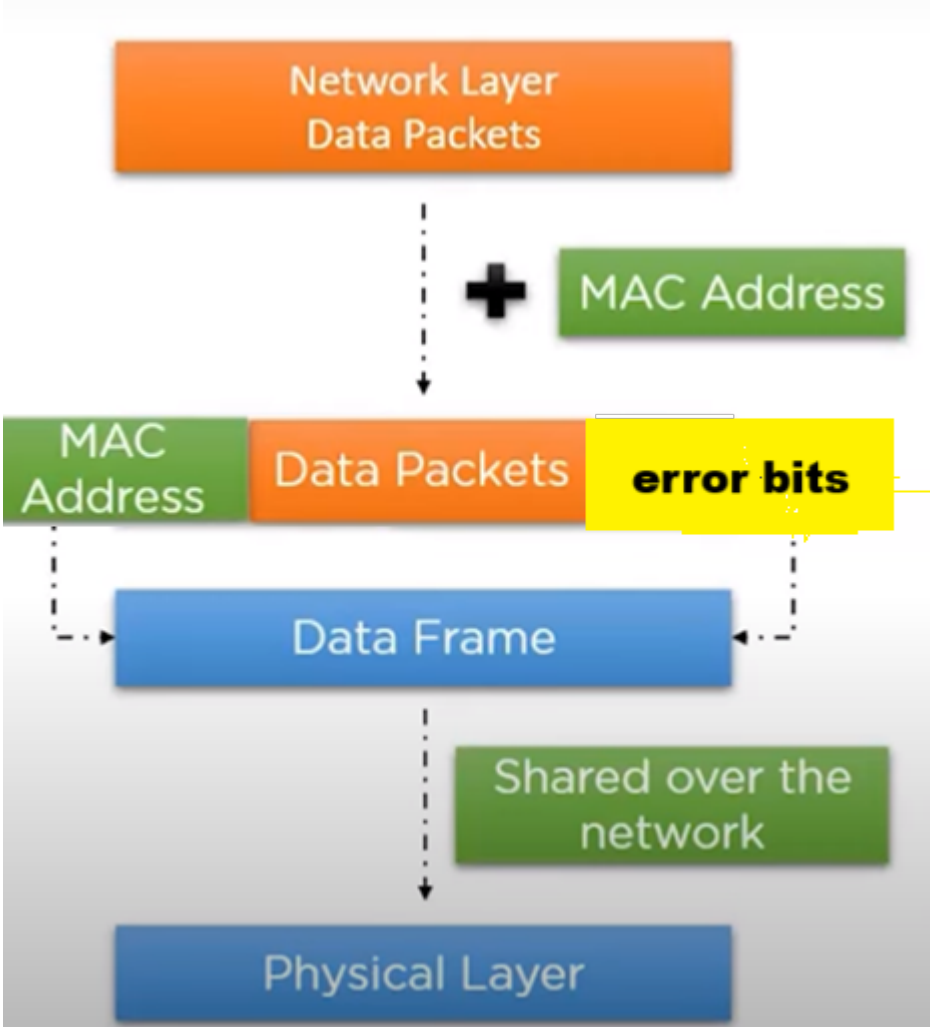


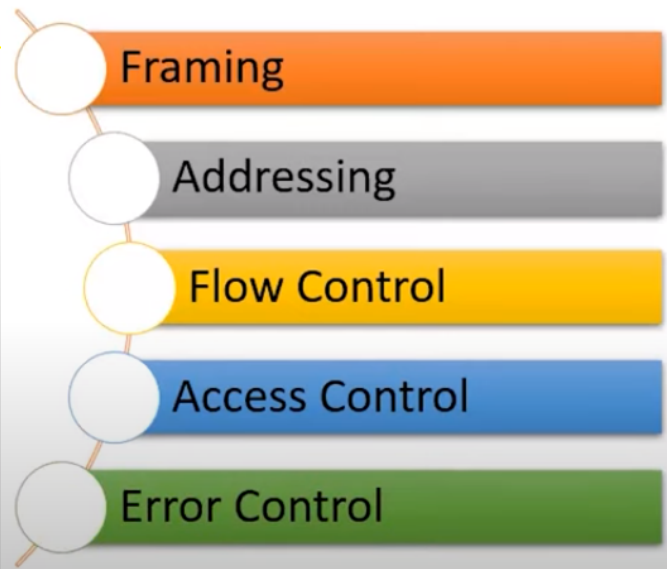
Data Link Layer

Framing, Error detection and Error correction, Vertical Redundancy Check (VRC), Longitudinal Redundancy Check (LRC), Hamming Distance, Hamming Code, Cyclic Redundancy Check (CRC), Stop and Wait Protocol, Sliding Window Protocol, Go-Back-n ARQ, Selective-Rject ARQ, HDLC, Channel Allocation, ALOHA Systems, CSMA Protocols, Collision Free Protocols

Data Link Layer



- ❑ Divided into two sublayers:
 - Medium Access control(MAC): Controls how devices establish the connection.
 - Logical Link Control(LLC): Identifies the address and provides flow control.

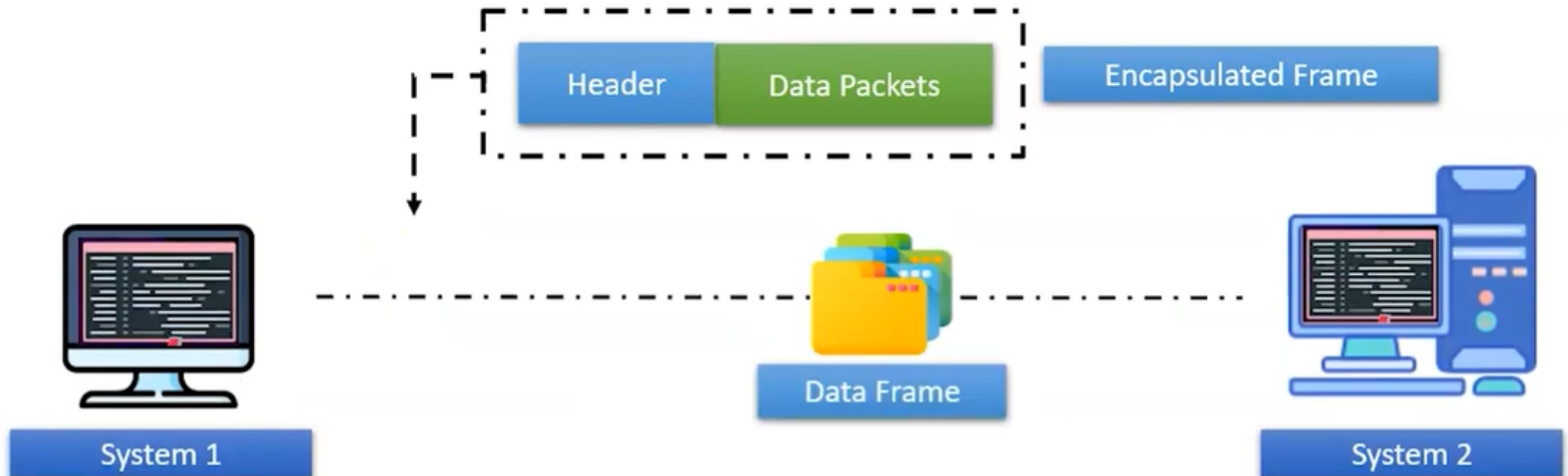


Service	Sublayer
Framing	MAC
Addressing	
Access Control	
Flow control	LLC
Error control	

Data Link Layer

1. Framing:

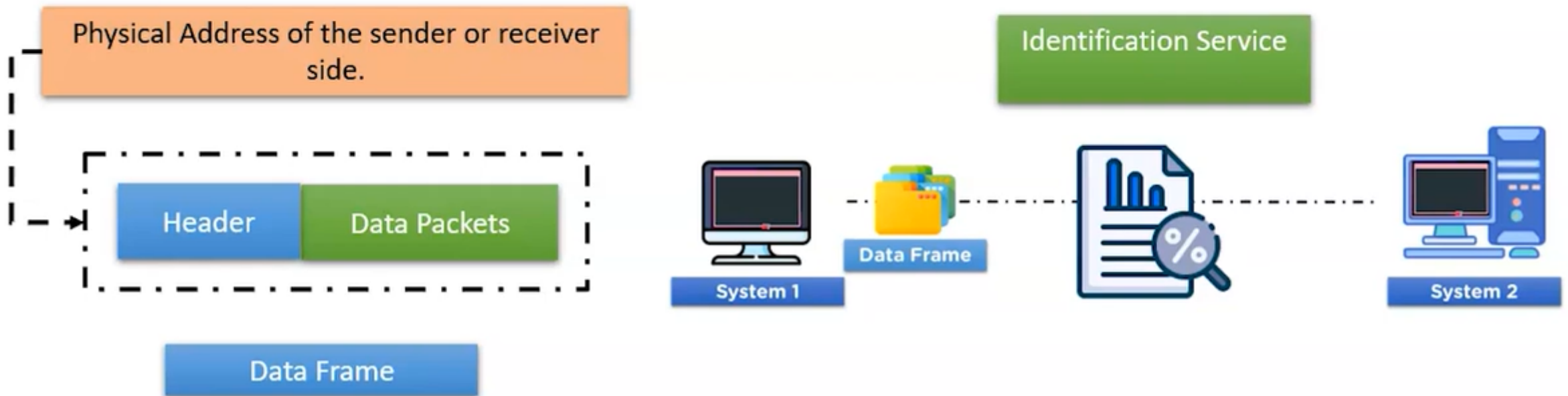
- The data packets received from the network layer are encapsulated in frames by the data-link layer for bit-to-bit sharing over the channel.
- It is also responsible for restructuring the framed data in the network model, and each data frame is different from the other.



Data Link Layer

2. Addressing:

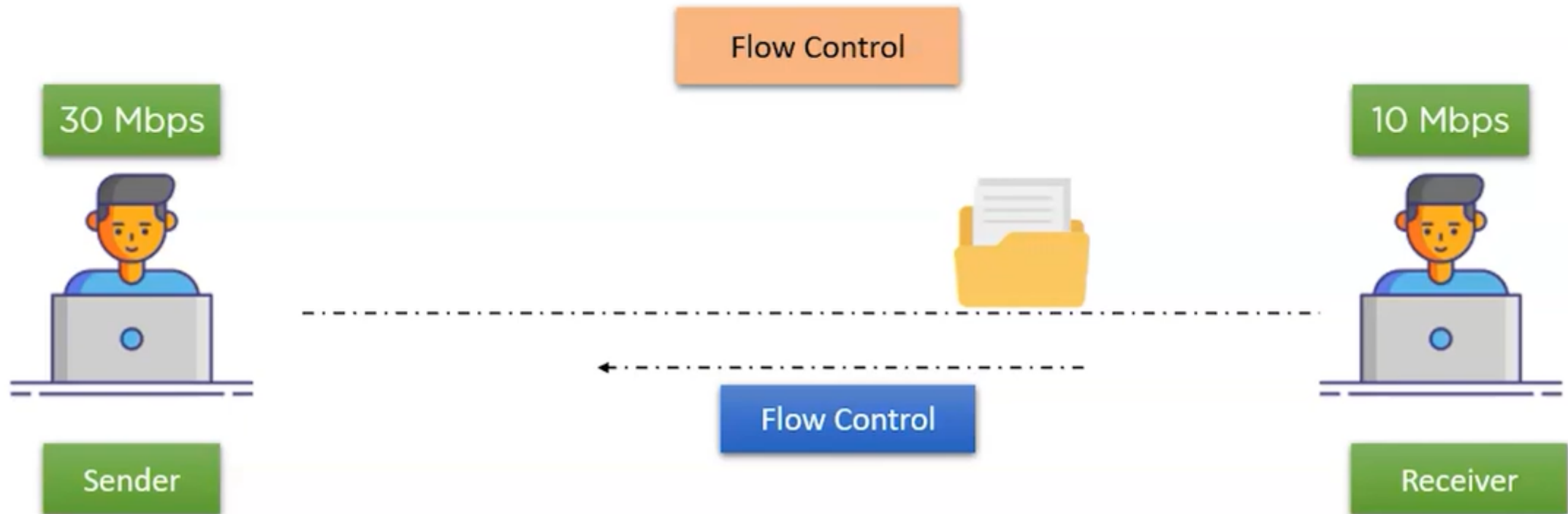
- ❖ The Data link layer appends the physical address in the header of the frame before sending it to physical layer.
- ❖ The physical address contains the address of the sender and receiver.
- ❖ In case the receiver happens to be on the same physical network as the sender; the receiver is at only one hop from the sender and the receiver address contains the receiver's physical address.
- ❖ In case the receiver is not directly connected to the sender, the physical address is the address of the next node where the data is supposed to be delivered.



Data Link Layer

3. Flow control

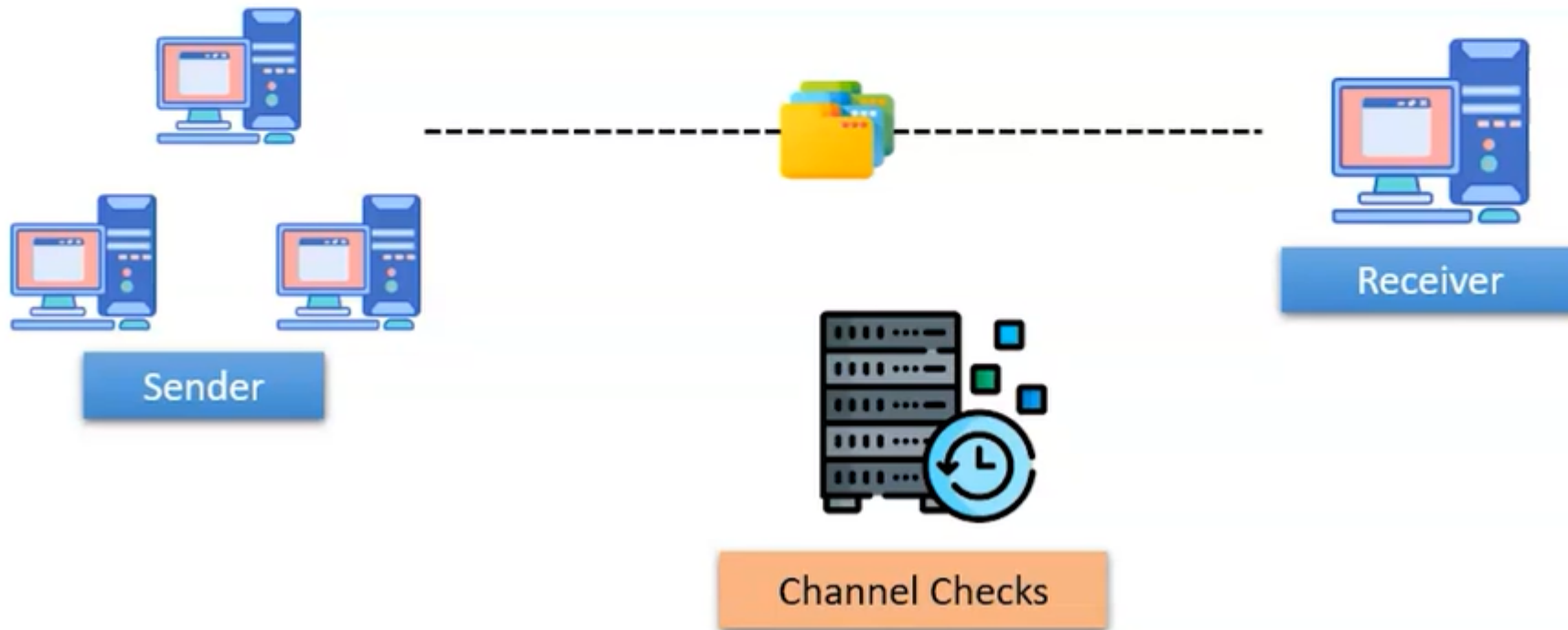
- ❖ The data link layer makes sure that the sender sends the data at a speed at which the receiver can receive it else if there is an overflow at the receiver side the data will be lost.
- ❖ The data link layer imposes flow control mechanism over the sender and receiver to avoid data loss at the receiver.



Data Link Layer

4. Access Control

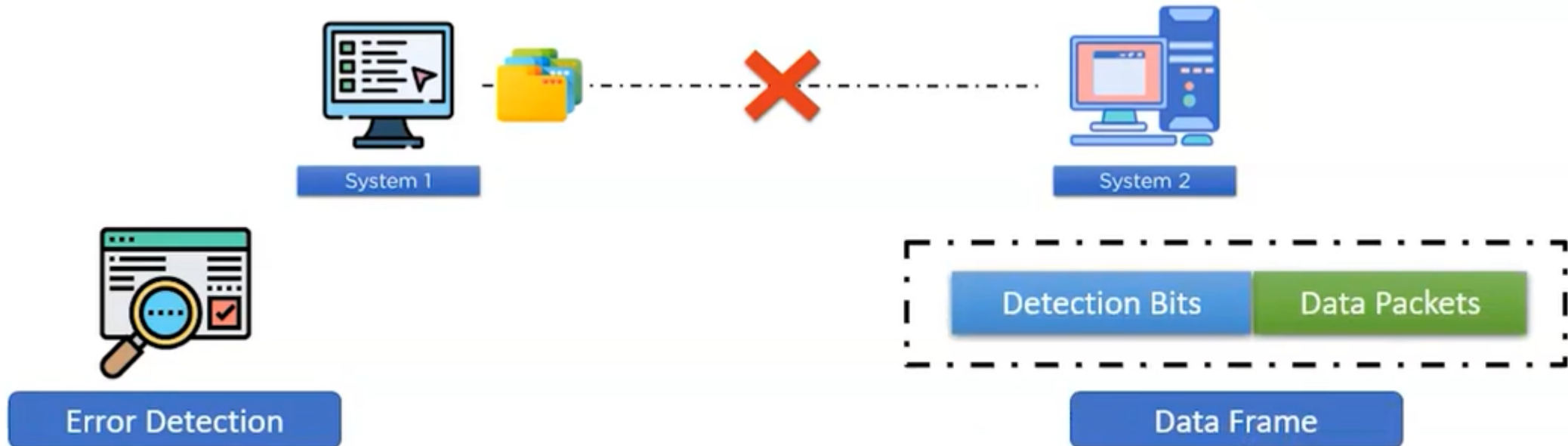
- ❖ The data link layer imposes access control mechanism to determine which device has right to send data in a multipoint connection scenario.



Data Link Layer

5. Error Control:

- During data transmission, due to noise or signal loss, errors might occur in the data being transmitted.
- To minimize such data error rate, the data-link layer performs error detection and correction techniques on the transmitted data.
- Error detection is done by adding detection bits in the trailer of the transmitted data, and the receiver side can check for any error in the received data.



Data Link Layer

- I. The Data Link layer adds reliability to the physical layer by providing error detection and correction mechanisms.
- II. On the sender side, the Data Link layer receives the data from Network Layer and divides the stream of bits into fixed size manageable units called as **Frames** and sends it to the physical layer. On the receiver side, the data link layer receives the stream of bits from the physical layer and regroups them into frames and sends them to the Network layer. This process is called **Framing**. It is shown in the figure below:

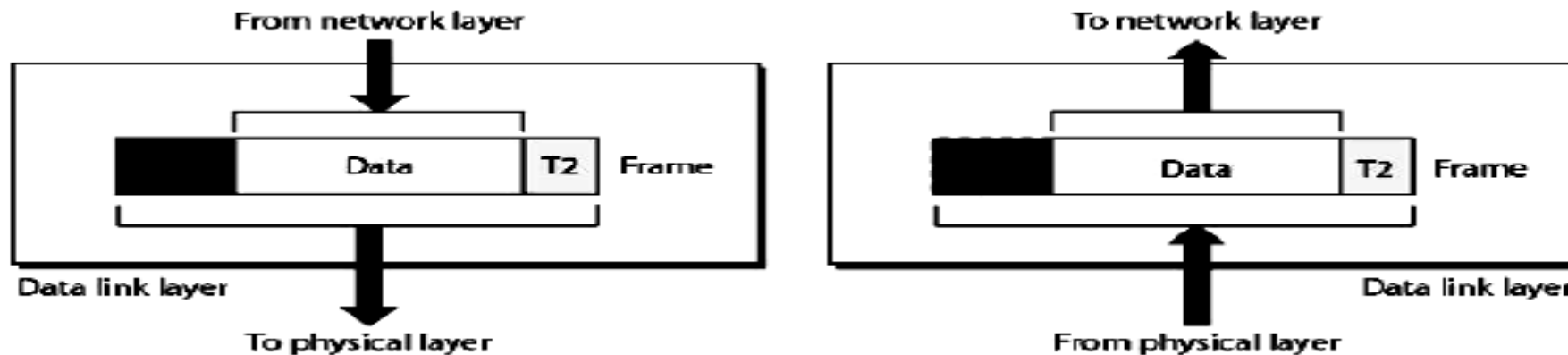
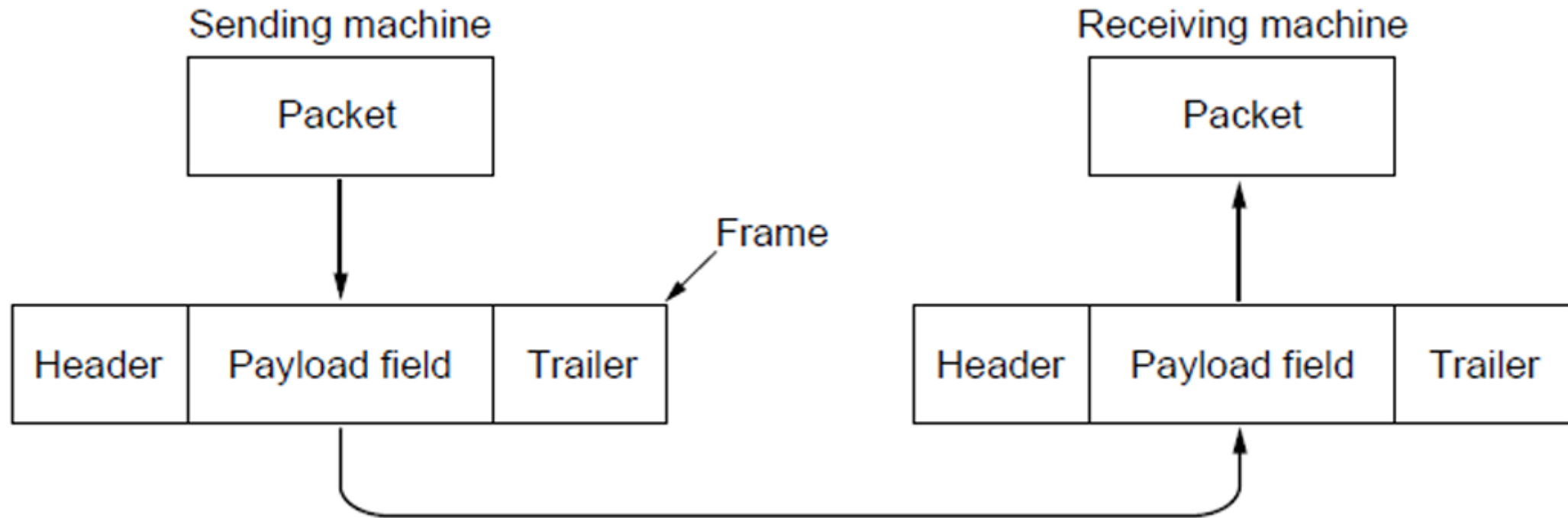


Fig: Data Link Layer: The process of Framing

Data Link Layer



Network layer – packets/data packets - stream of bits / Divide the stream of bits into fixed size manageable units– **payload field**
101011101010101010101011101010101010111

Header – physical address/MAC address of sender and receiver

Trailer – error detection bits

3.2 Framing and its types

Data Link Layer:

- The two main functions of the data link layer are **data link control** and **media access control**.
- **Data link control** functions include **framing**, flow and error control, and software implemented protocols that provide smooth and reliable transmission of frames between nodes.
- The second function of the data link layer is **media access control**, or how to share the link.

Framing:

- Frames are nothing but manageable group of bits to be transmitted.
- Bits are packed into frames, so that each frame is distinguishable from another.
- Framing in the data link layer separates a message from one source to a destination, or from other messages to other destinations, by adding a sender address and a destination address.

3.2 Framing and its types

Types of Framing:

1. Fixed-Size Framing:

Frames can be of fixed or variable size. In fixed-size framing, there is no need for defining the boundaries of the frames; the size itself can be used as a delimiter. An example of this type of framing is the ATM wide-area network, which uses frames of fixed size called cells.

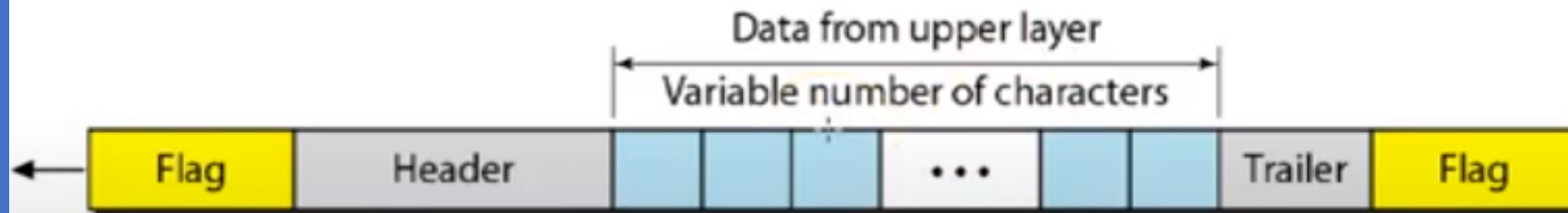
2. Variable-Size Framing:

Variable-size framing, prevalent in local area networks in which we need a way to define the end of the frame and the beginning of the next. Two approaches are used for this purpose:

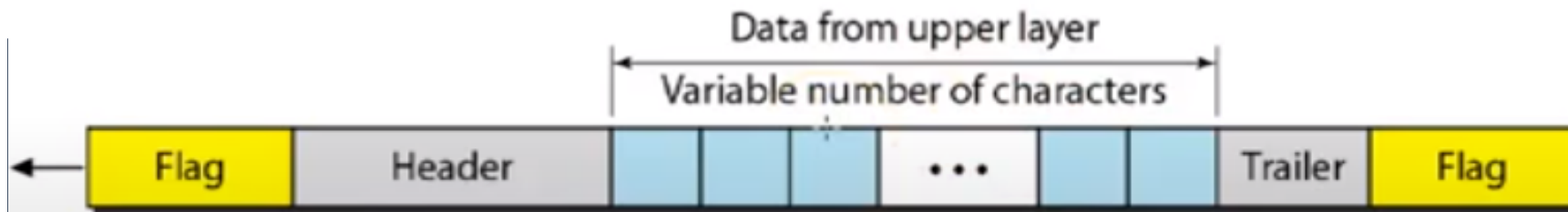
3.2 Framing and its types

Character-Oriented Protocol

- In a character-oriented protocol, data to be carried are 8-bit characters from a coding system such as ASCII.
- The header, which normally carries the source and destination addresses and other control information, and the trailer, which carries error detection or error correction redundant bits, are also multiples of 8 bits.
- To separate one frame from the next, an 8-bit (1 byte) flag is added at the beginning and the end of a frame. The flag, composed of protocol dependent special characters, signals the start or end of a frame. Figure shows the format of a frame in a character-oriented protocol.



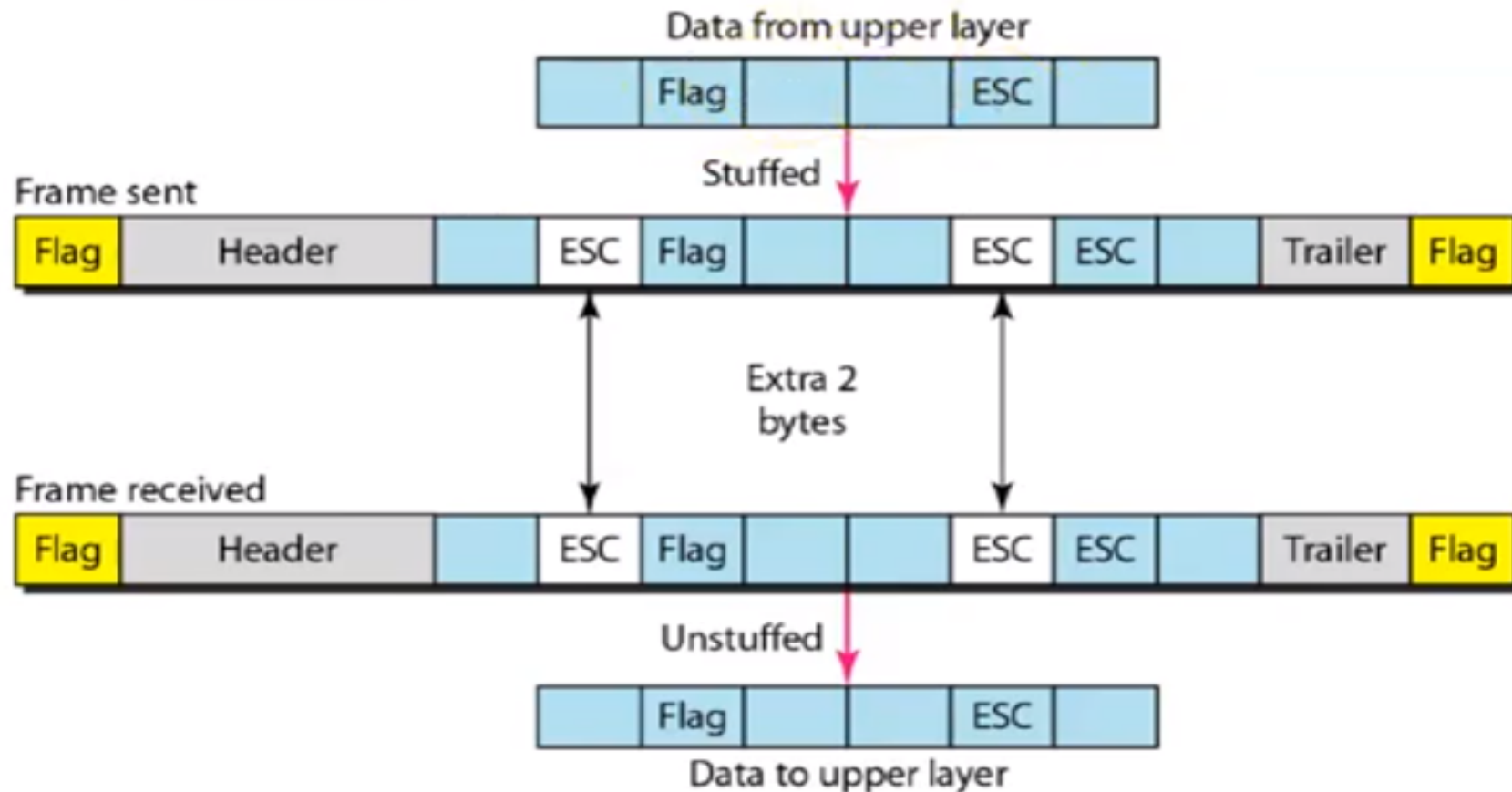
3.2 Framing and its types



- Any pattern used for the flag could also be part of the information. If this happens, the receiver, when it encounters this pattern in the middle of the data, thinks it has reached the end of the frame.
- To fix this problem, a byte-stuffing (or character stuffing) strategy is added to character-oriented framing in which a special byte is added (stuffed) to the data section of the frame when there is a character with the same pattern as the flag.
- This special extra byte is usually called the escape character (ESC), which has a predefined bit pattern. Whenever the receiver encounters the ESC character, it removes it from the data section and treats the next character as data, not a delimiting flag.
- Byte stuffing by the escape character allows the presence of the flag in the data section of the frame, but it creates another problem that if the text contains one or more escape characters followed by a flag then the receiver removes the escape character, but keeps the flag, which is incorrectly interpreted as the end of the frame.

3.2 Framing and its types

- To solve this problem, the escape characters that are part of the text must also be marked by another escape character. In other words, if the escape character is part of the text, an extra one is added to show that the second one is part of the text. Figure shows the situation.



3.2 Framing and its types

Limitations of Character-Oriented Protocol:

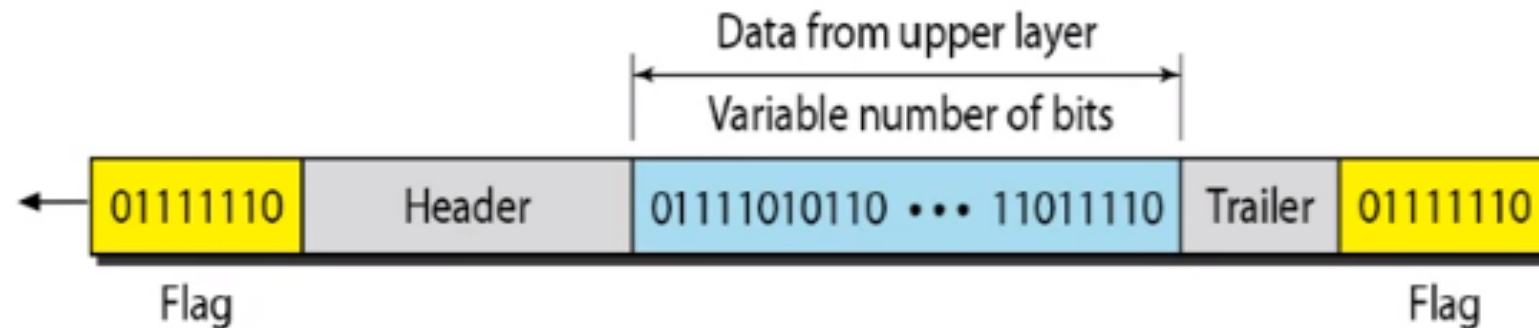
- Character-oriented framing is suitable when only text is exchanged by the data link layers. It is not useful for sending other types of information such as graphs, images, audio, and video or other multimedia data.
- Character-oriented protocols present another problem in data communications. The universal coding systems in use today, such as Unicode, have 16-bit and 32-bit characters that conflict with 8-bit characters.

Thus, we can say that in general, the tendency is moving toward the bit-oriented protocols.

3.2 Framing and its types

Bit-Oriented Protocol:

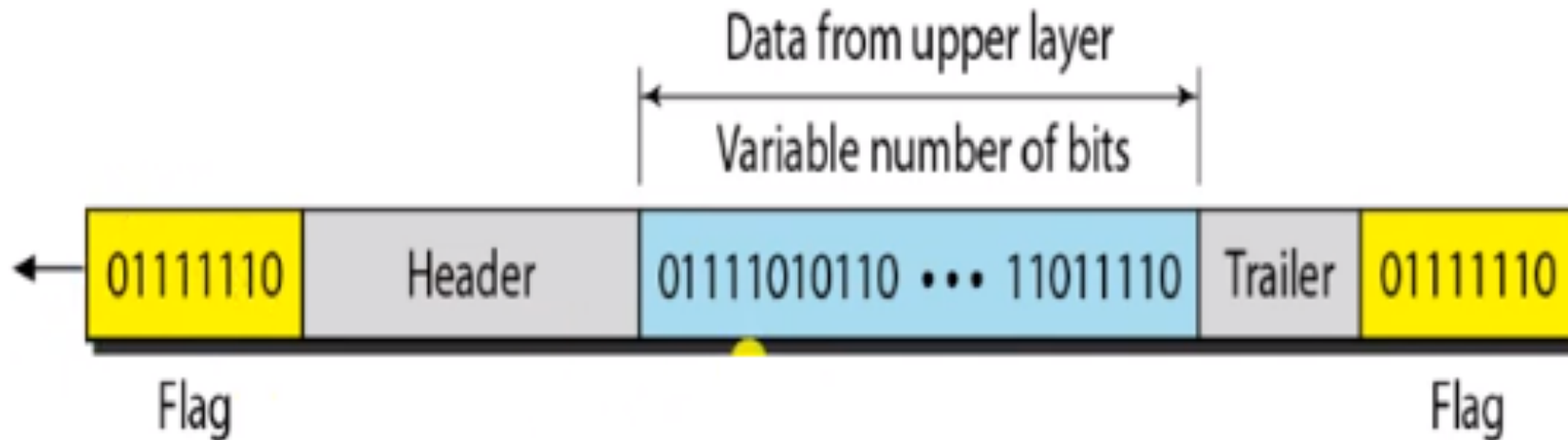
- In a bit-oriented protocol, the data section of a frame is a sequence of bits to be interpreted by the upper layer as text, graphic, audio, video, and so on.
- In addition to headers (and possible trailers), we still need a delimiter to separate one frame from the other.
- Most protocols use a special 8-bit pattern flag 01111110 as the delimiter to define the beginning and the end of the frame, as shown in Figure.



- This flag can create the same type of problem as in the byte-oriented protocols.
- It means, if the flag pattern appears in the data, we need to inform the receiver that this is not the end of the frame.

3.2 Framing and its types

- Most protocols use a special 8-bit pattern flag 01111110 as the delimiter to define the beginning and the end of the frame, as shown in Figure.

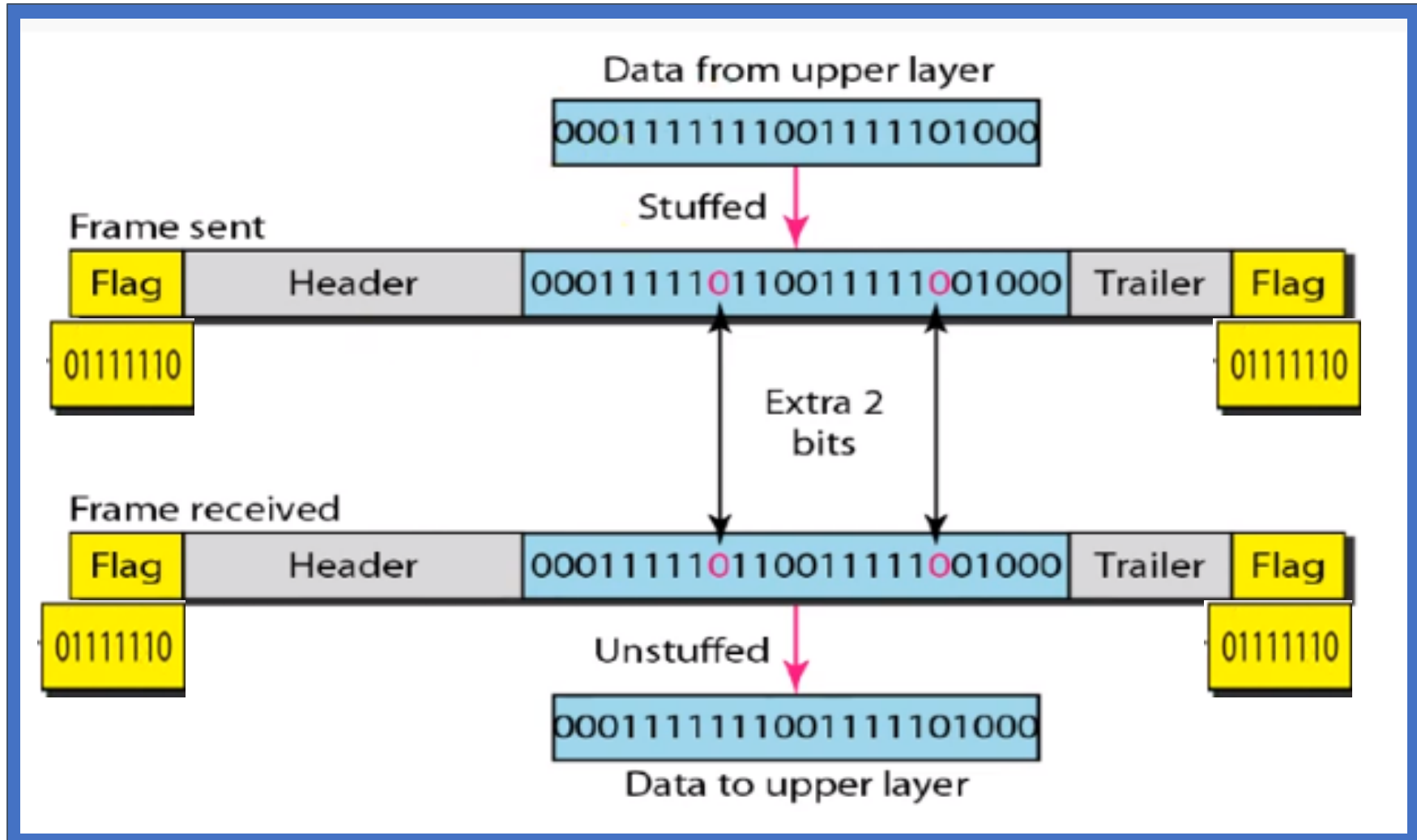


- This flag can create the same type of problem as in the byte-oriented protocols.
- It means, if the flag pattern appears in the data, we need to inform the receiver that this is not the end of the frame.

3.2 Framing and its types

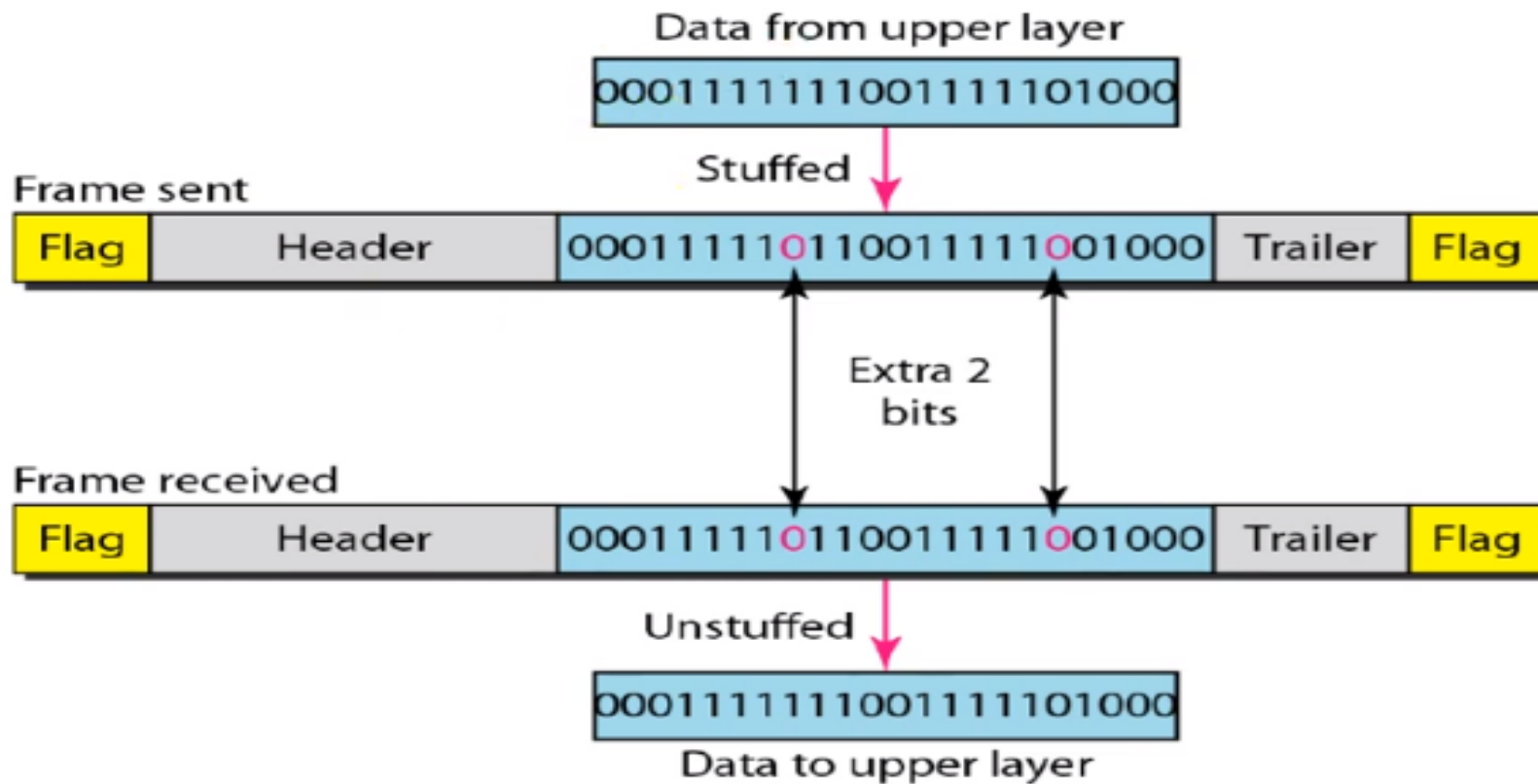
- We do this by stuffing 1 single bit (instead of 1 byte) to prevent the pattern from looking like a flag.
- The strategy is called bit stuffing. In bit stuffing, if a 0 and five consecutive 1 bits are encountered, an extra 0 is added. This extra stuffed bit is eventually removed from the data by the receiver.
- The extra bit is added after one 0 followed by five 1s regardless of the value of the next bit. This guarantees that the flag field sequence does not inadvertently appear in the frame.
- Figure shows bit stuffing at the sender and bit removal at the receiver. Even if we have a 0 after five 1s, we still stuff a 0. The 0 will be removed by the receiver.

3.2 Framing and its types



3.2 Framing and its types

This means that if the flag like pattern 01111110 appears in the data, it will change to 011111010 (stuffed) and is not mistaken as a flag by the receiver. The real flag 01111110 is not stuffed by the sender and is recognized by the receiver.



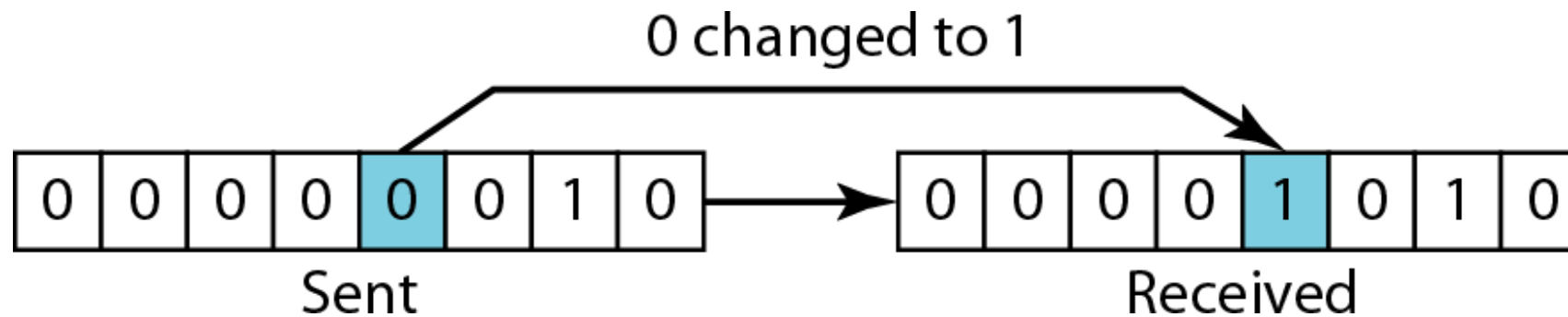
**Data can be corrupted
during transmission.**

**Some applications require that
errors be detected and corrected.**

Types of Error

Single-bit error

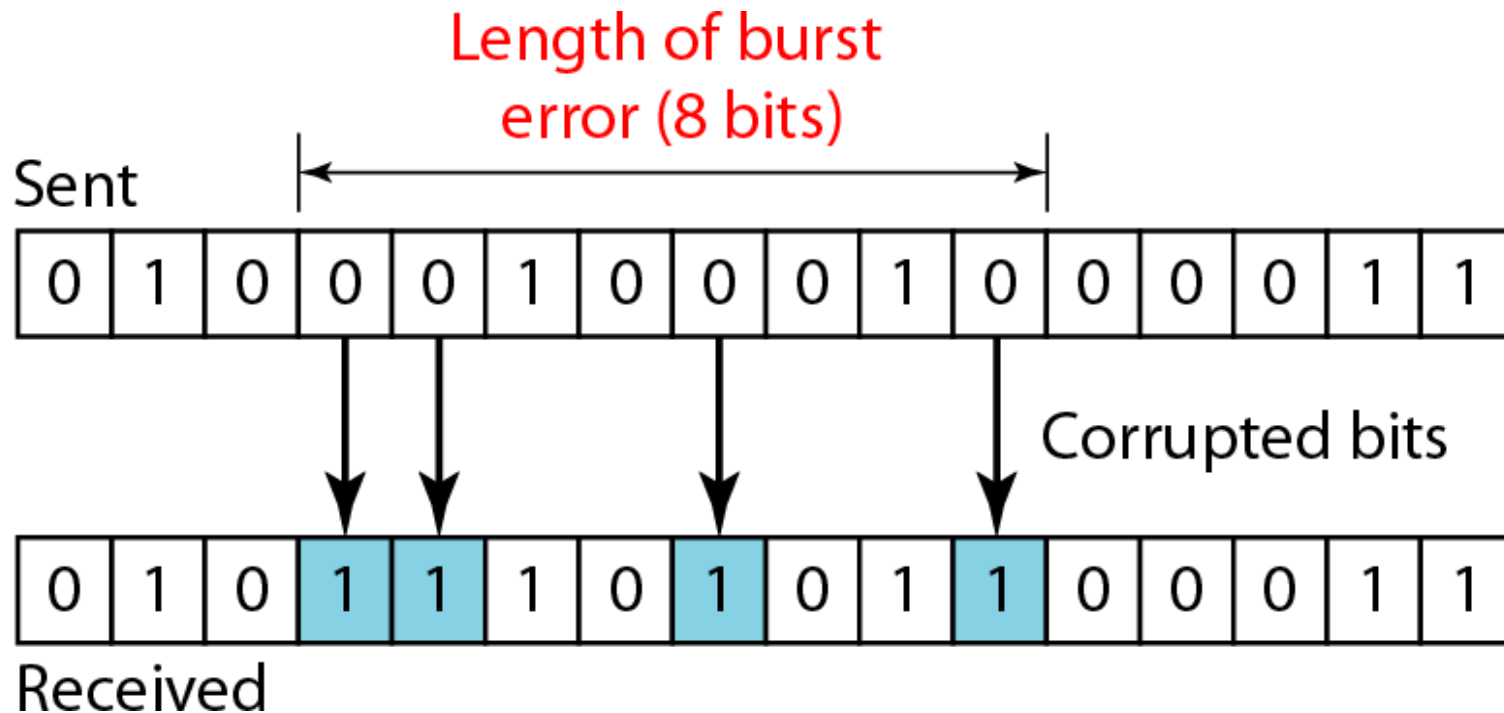
In a single-bit error, only 1 bit in the data unit has changed.



Types of Error

Burst error

A burst error means that 2 or more bits in the data unit have changed.



- ❖ Note that a burst error does not necessarily mean that the errors occur in consecutive bits.
- ❖ The length of the burst is measured from the first corrupted bit to the last corrupted bit.
- ❖ Some bits in between may not have been corrupted.



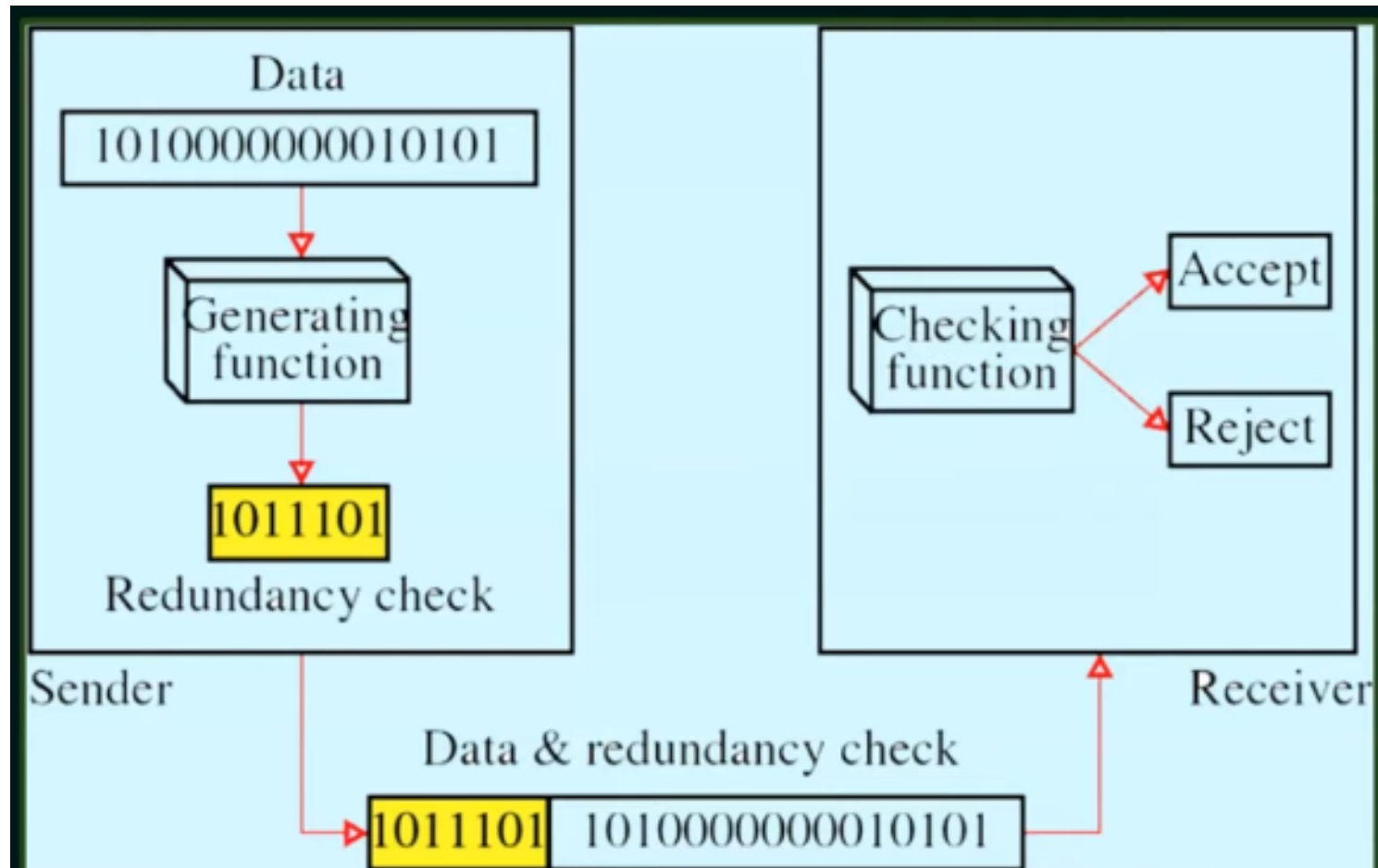
Error Detection

- **Redundancy** : is the central concept in detecting & correcting errors.
- We need to send some extra bits with our data. These redundant bits are added by the sender and removed by the receiver .

To detect or correct errors, we need to send extra (redundant) bits with data.

Data	Extra bits (Redundant bits)
------	-----------------------------

Figure *The structure of encoder and decoder*



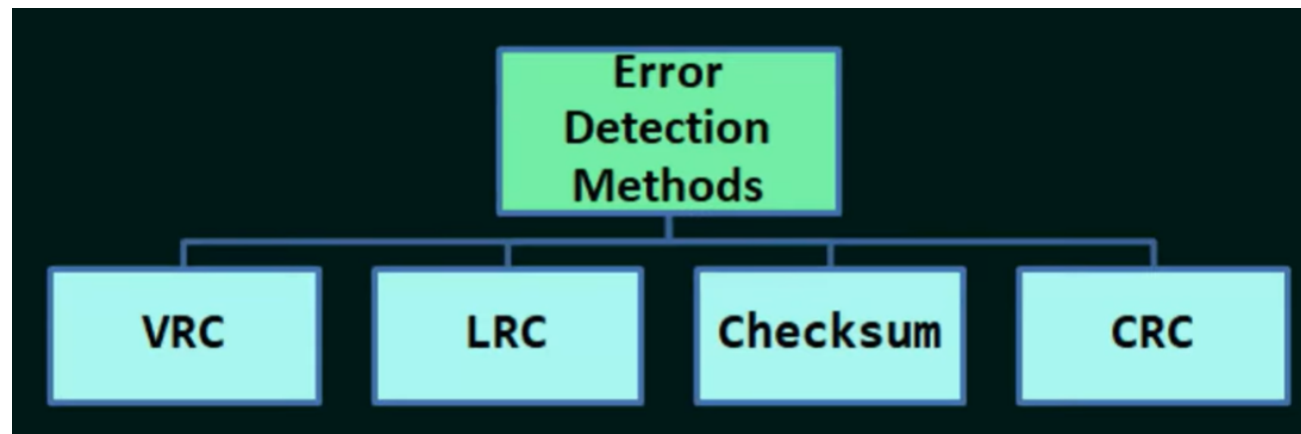
Detection Versus Correction

- In error **detection** , we are looking **only** to see **if** any error has occurred. A single-bit error is the same for us as a burst error.
- In error **correction** , we need to **know** the exact number of bits that are corrupted and more importantly, their location in the message.
- **Note:** correction of errors is more difficult than the detection

Error Detection Techniques

Four types of redundancy checks are used in data communications. They are:

1. Vertical Redundancy Check (VRC)
2. Longitudinal Redundancy Check (LRC)
3. Checksum
4. Cyclic Redundancy Check (CRC)



VRC

- ★ Vertical Redundancy Check.
- ★ It is also called as parity check.

Data

1100001

VRC

Parity generator will count number of 1s in data unit and will add parity bit.

★ Vertical Redundancy Check.

★ It is also called as parity check.

VRC: 1; if odd number of 1's

VRC: 0; if even number of 1's

Data

1100001



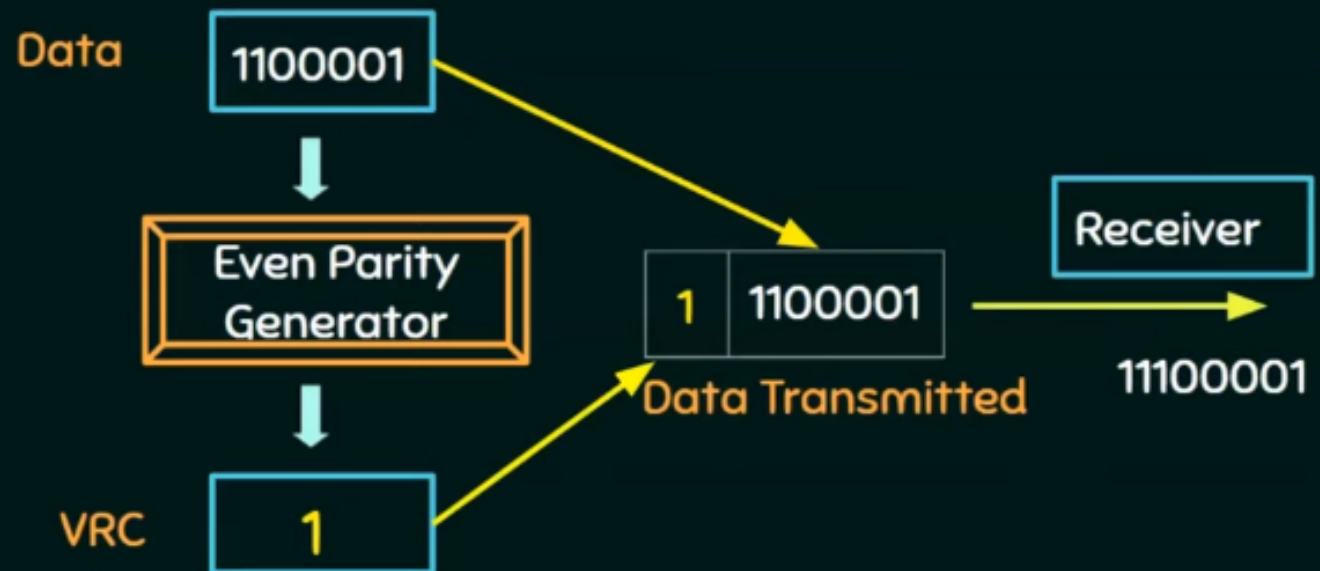
Even Parity
Generator



1

VRC

- ★ Vertical Redundancy check.
- ★ It is also called as parity check.



VRC

- ★ It can detect single bit error.
- ★ It can detect burst error only if the number of errors is odd.

Sender: 11100001 → Transmission Error 10100001 → Receiver rejects this data.

Sender: 11100001 → Transmission Error 10100101 → Receiver accepts this data.

VRC

Find the even parity bit (VRC) for the following message stream:

1110110

1101111

1110010

LRC

- ★ Longitudinal Redundancy Check.
- ★ In LRC, a block of bits is organized in rows and columns.
- ★ a.k.a Two Dimensional parity.
- ★ The parity bit is calculated for each column and sent along with the data.
- ★ The block of parity acts as the redundant bits.

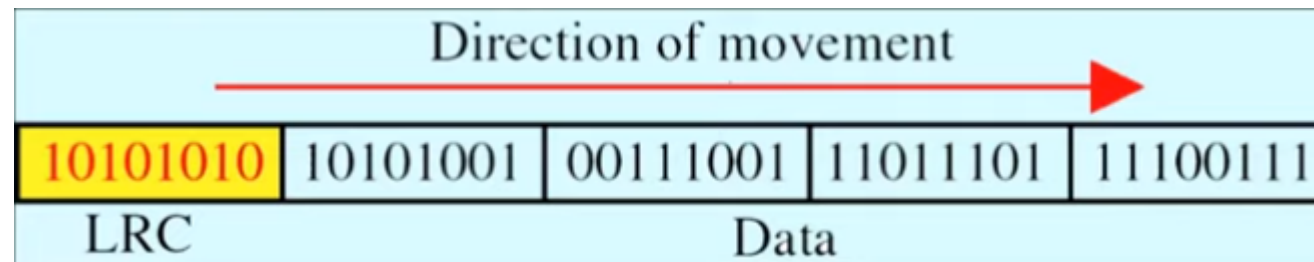
LRC

Find the LRC for the data blocks 1100111 11011101 00111001 10101001 and determine the data that is transmitted?

Odd no. of 1's	1
Even no. of 1's	0

LRC →

1	1	1	0	0	1	1	1
1	1	0	1	1	1	0	1
0	0	1	1	1	0	0	1
1	0	1	0	1	0	0	1
1	0	1	0	1	0	1	0



LRC

PERFORMANCE OF LRC

- ★ LRC increases the likelihood of detecting burst errors.
- ★ If two bits in one data units are damaged and two bits in exactly the same positions in another data unit are also damaged, the LRC checker will not detect an error.

1	1	1	0	0	1	1	1
1	1	0	1	1	1	0	1
0	0	1	1	1	0	0	1
1	0	1	0	1	0	0	1
1	0	1	0	1	0	1	0

LRC

Find the LRC for the data blocks 0110111 10101001 01101001 10101010 and determine the data that is transmitted.

Cyclic Redundancy Check

- It is one of the error detection technique
- Steps for finding the redundant bits

1. Find the length of the divisor 'L'.
2. Append 'L-1' bits to the original message.
3. Perform binary division operation.
4. Remainder of the division = CRC.

Note:

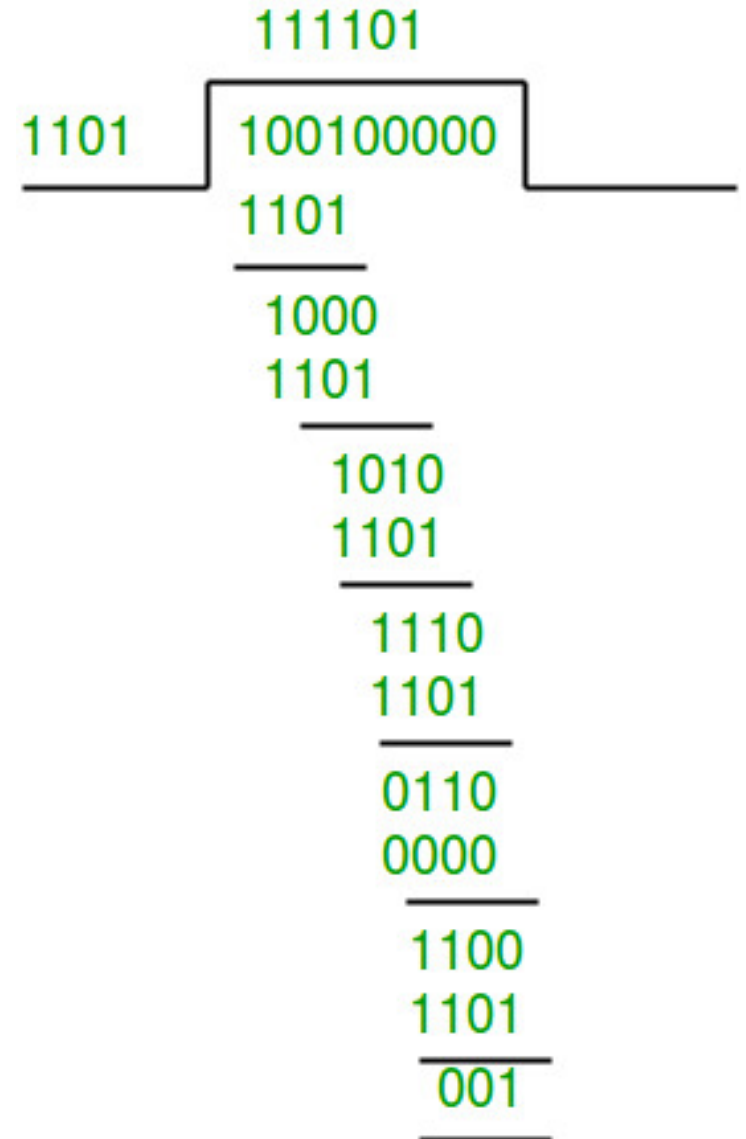
The CRC must be of L-1 bits.

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Find the CRC for the data blocks 100100 with the divisor 1101?

Cyclic Redundancy Check

Find the CRC for the data blocks 100100 with the divisor 1101?



The remainder is **001** and

hence the transmitted data is **100100001**.

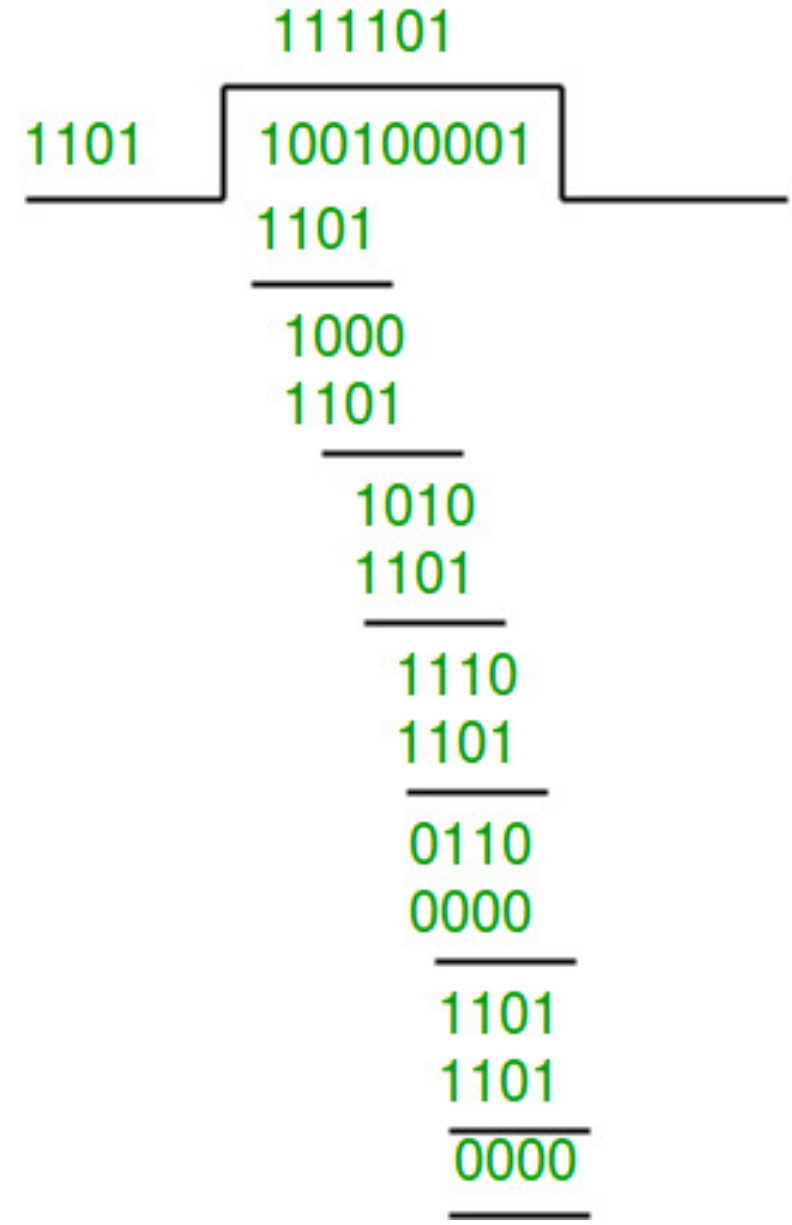
Cyclic Redundancy Check

Receiver Side:

Received bits 100100001

The remainder is all zeros.

Hence, the data received has no error.



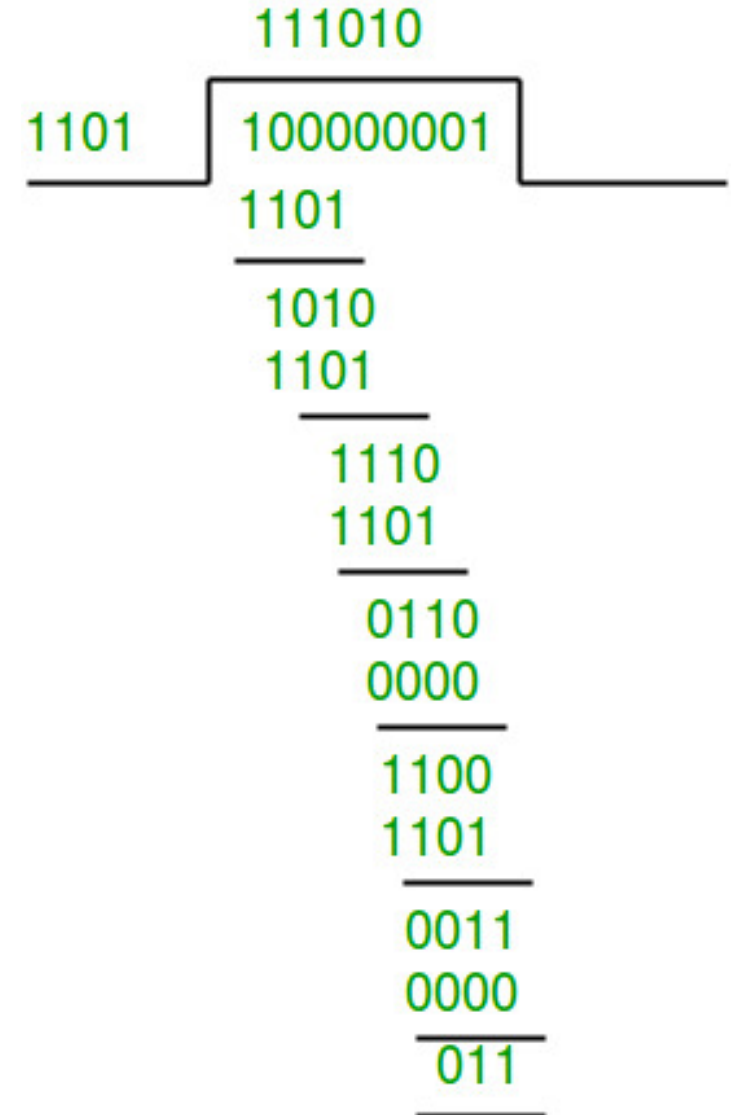
Cyclic Redundancy Check

Receiver Side:

Received bits 100000001

The remainder is not all zeros.

Hence, the error is detected at the receiver side.



Cyclic Redundancy Check

Exercise:

Find the CRC for 110010101 with the divisor x^3+x^2+1 ?

Error Correction

- There are two main methods of error correction.
- **Forward error correction** is the process in which the receiver tries to correct the error using error correction techniques.
- **Correction by retransmission** is a technique in which the receiver detects the occurrence of an error and asks the sender to resend the message. Resending is repeated until a message arrives that the receiver believes is error-free (usually, not all errors can be detected)

What is Hamming Code?

The “Hamming Code” method is a network technique designed to **detect errors and correct bits** which are transmitted during data exchange, first implemented by **R.W. Hamming**.



This network method is implemented because, during data transmission, many data bits may get lost or corrupted.

Terms Related to Hamming Code

Redundant Bits:

- They are the extra binary bits that are added explicitly to the original data, to prevent damage to the transmitted data and are also used to recover original data, in case of damage.
- To determine the number of “redundant bits”, below mentioned formula is applied:

$$2^r \geq m+r+1,$$

Where,

m – “data bits”

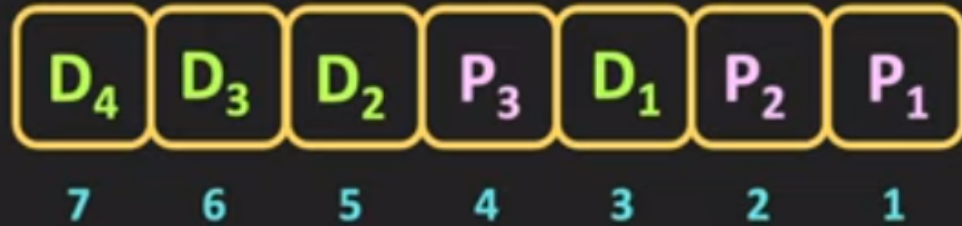
r – “redundant bits”

For Example: In case **m=4**, find the value of r=?

- $2^r \geq 4+r+1$
- $2^r \geq r+5$



Hamming Code(7,4)



Parity Bits at the position of $2^0, 2^1, 2^2$ -----

Bit
Position In Binary

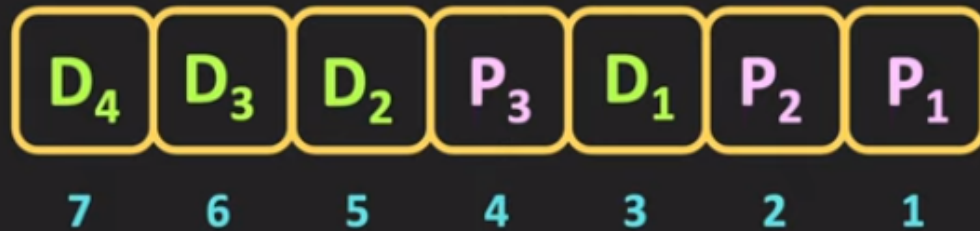
			P1
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

P_1 represents the parity of all the bit positions whose LSB is 1

P_1 3, 5 and 7 even

$$P_1 = D_1 \oplus D_2 \oplus D_4$$

Hamming Code(7,4)



Parity Bits at the position of $2^0, 2^1, 2^2$ -----

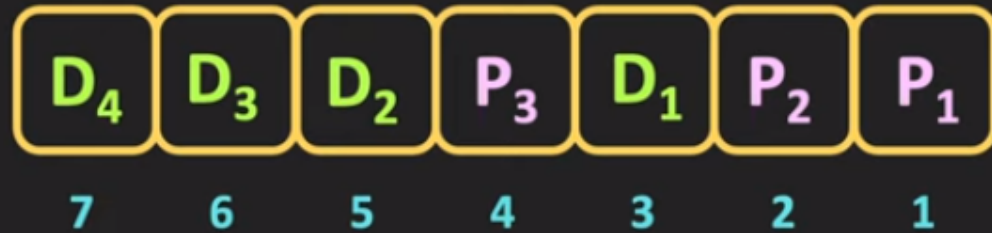
Bit Position	In Binary
	P2
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1

P_2 represents the parity of all the bit positions whose second LSB is 1

P_2 3 6 and 7 even

$$P_2 = D_1 \oplus D_3 \oplus D_6$$

Hamming Code(7,4)



Parity Bits at the position of $2^0, 2^1, 2^2$ ----

Bit Position	In Binary
	P3
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1

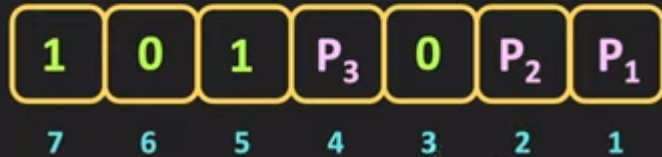
P_3 represents the parity of all the bit positions whose MSB is 1

P_3 5 6 and 7 even

$$P_3 = D_2 \oplus D_3 \oplus D_4$$

Hamming Code(7,4)

Data Bits – 1 0 1 0



Parity Bit Calculation:

P₁ 3, 5 and 7 even

P₁ 0 1 1

0

P₂ 3, 6 and 7 even

P₂ 0 0 1

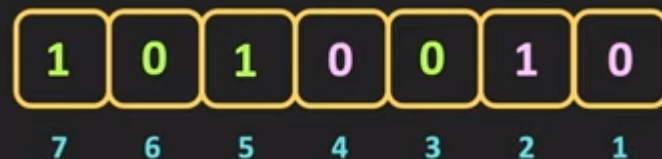
1

P₃ 5, 6 and 7 even

P₃ 1 0 1

0

Transmitted Bit



Error Detection – Hamming Code

Transmitted Bits – 1 0 1 0 0 1 0

Received Bits – 1 1 1 0 0 1 0

Parity Bits		Calculated Parity
$P_1 - 3,5,7 - P_1, D_1, D_2, D_4$	$P_1, D_1, D_2, D_4 - 0\ 0\ 1\ 1$	0
$P_2 - 3,6,7 - P_2, D_1, D_3, D_4$	$P_2, D_1, D_3, D_4 - 1\ 0\ 1\ 1$	1
$P_3 - 5,6,7 - P_3, D_2, D_3, D_4$	$P_3, D_2, D_3, D_4 - 0\ 1\ 1\ 1$	1

Calculated Parity bits are not zero. It indicates that there is error in the received bits – Error is detected

Error Detection – Hamming Code

Transmitted Bits – 1010010

Received Bits – 1010010

$D_1 = 0$, $D_2 = 1$, $D_3 = 0$, $D_4 = 1$

Parity Bits	Calculated Parity	Received Parity	Check Received = Calculated
$P_1 = 3,5,7 - D_1, D_2, D_4 = 0,1,1$	0	0	0
$P_2 = 3,6,7 - D_1, D_3, D_4 = 0, 0, 1$	1	1	0
$P_3 = 5,6,7 - D_2, D_3, D_4 = 1,0,1$	0	0	0

The received and calculated parity bits are same. It indicates that there is no error in the received bits

Error Correction – Hamming Code

Transmitted Bits – 1 0 1 0 0 1 0

Received Bits – 1 1 1 0 0 1 0

Parity Bits	Calculated Parity	Received Parity	Check Calculated Parity = Received Parity (Syndrome)
$P_1 - 3,5,7 - D_1, D_2, D_4 - 0, 1, 1$	0	0	0
$P_2 - 3,6,7 - D_1, D_3, D_4 - 0, 1, 1$	0	1	1
$P_3 - 5,6,7 - D_2, D_3, D_4 - 1, 1, 1$	1	0	1

The received and calculated parity bits are not same. It indicates that there is an error in the received bits

Syndrome – $(110)_2 - 6$ indicated the error bit position

6th bit position value is 1 so to correct the error it should be flipped

Corrected data is 1010010

Hamming Distance

The Hamming distance between two words is the number of differences between corresponding bits.

The Hamming distance can easily be found if we apply the XOR operation ($\oplus$) on the two words and count the number of **1's** in the result.

Hamming Distance

Let us find the Hamming distance between two pairs of words.

1. The Hamming distance $d(000, 011)$ is 2 because

$000 \oplus 011$ is 011 (two 1s)

2. The Hamming distance $d(10101, 11110)$ is 3 because

$10101 \oplus 11110$ is 01011 (three 1s)

Minimum Hamming Distance

The minimum Hamming distance is the smallest Hamming distance between all possible pairs in a set of words.

	8	4	2	1	BCD
0	0	0	0	0	1
1	0	0	0	1	
2	0	0	1	0	
3	0	0	1	1	
4	0	1	0	0	2
5	0	1	0	1	
6	0	1	1	0	
7	0	1	1	1	
8	1	0	0	0	4
9	1	0	0	1	

Minimum Hamming distance = $d_{\min} = 1$

Minimum Hamming Distance of Hamming Code (n,k)

□ $d_{\min} = n - k + 1$

Where, n is the total number of bits
k is the number bits in the data

Minimum Hamming Distance

d bits of error detection

Minimum Hamming Distance $d_{\min} = d + 1$

d bits of error correction

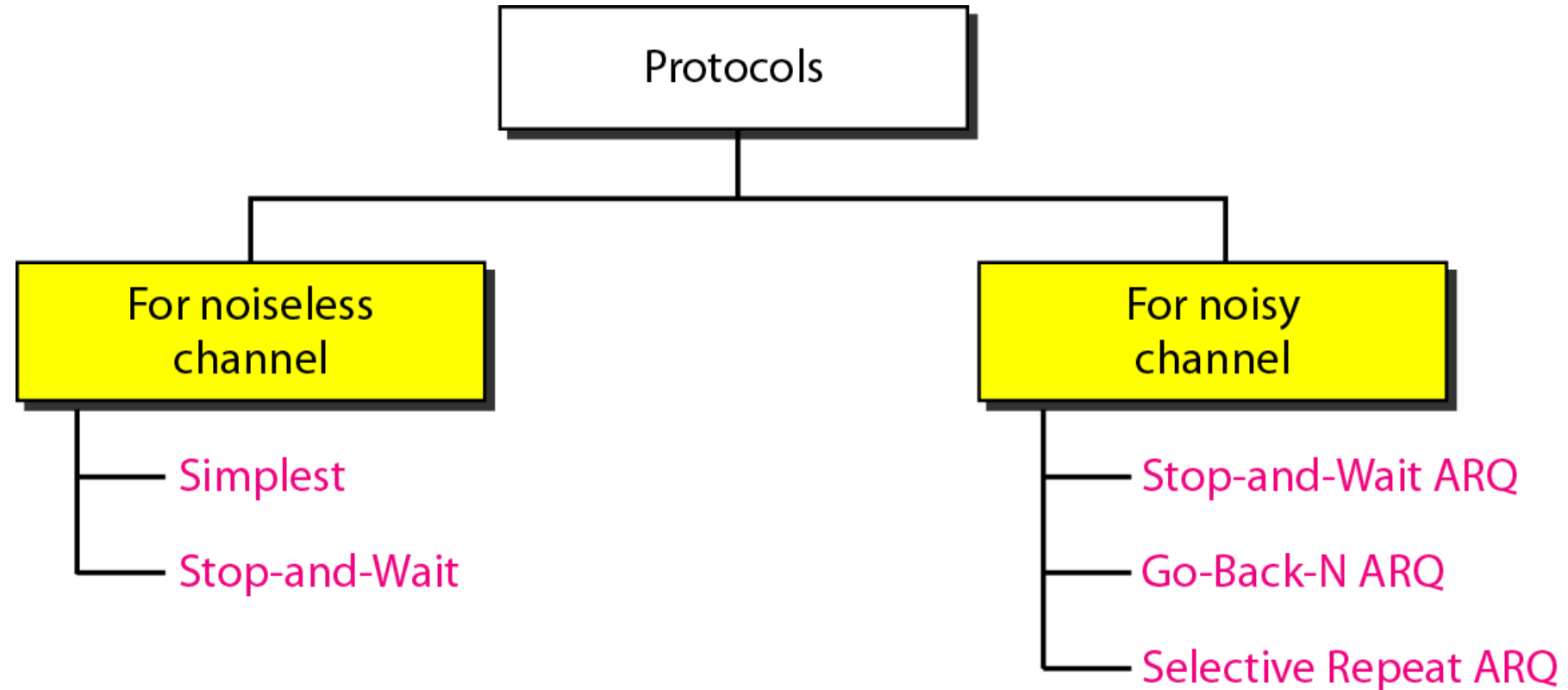
Minimum Hamming Distance $d_{\min} = 2d + 1$

1. If the minimum hamming distance is 9 then how many errors can be detected and corrected by those codewords?
2. For (11,7) hamming code, how many errors can be corrected and detected?

PROTOCOLS

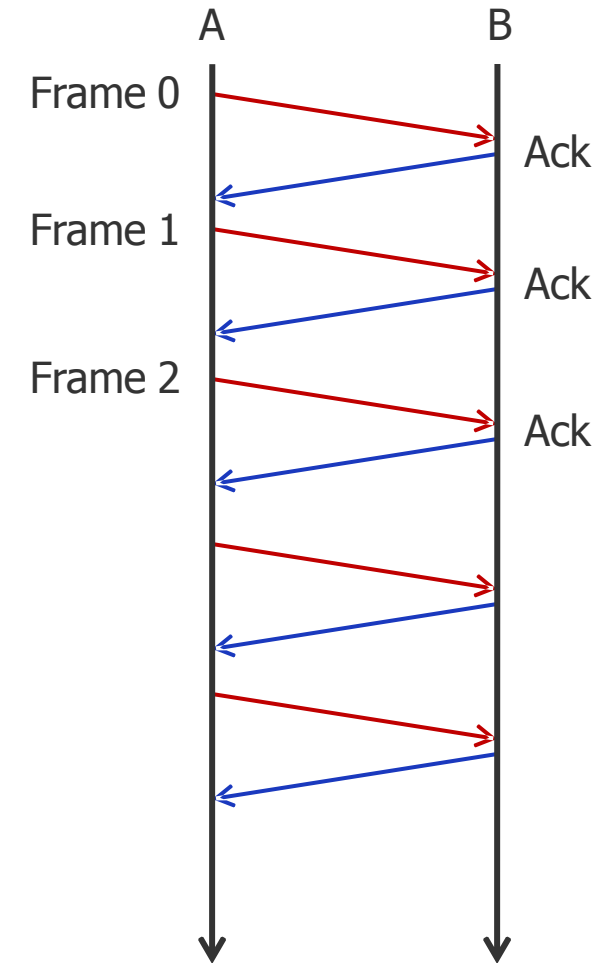
In data link layer the flow control is achieved through the protocols. The protocols are normally the set of rules and implemented using software by using one of the common programming languages.

Taxonomy of protocols



Data Link Layer Protocol - Stop and Wait Protocol

- Stop-and-wait protocol provides data transmission with flow control facilities but without error control facilities.
- The sender sends a one data frame at a time and **w a i t s**, until receiving an **acknowledgement** from the receiver.



Data Link Layer Protocol - Stop and Wait Protocol

Problems of stop-and-wait protocol

1. Problem due to lost data

- Sender waits for ack for an infinite amount of time
- Receiver waits for data an infinite amount of time

2. Problems due to lost ACK

- Sender waits for an infinite amount of time for the ACK

3. Problems due to delayed ACK/data

- After timeout on sender side, a delayed ack might be wrongly considered as ack of some other data packet

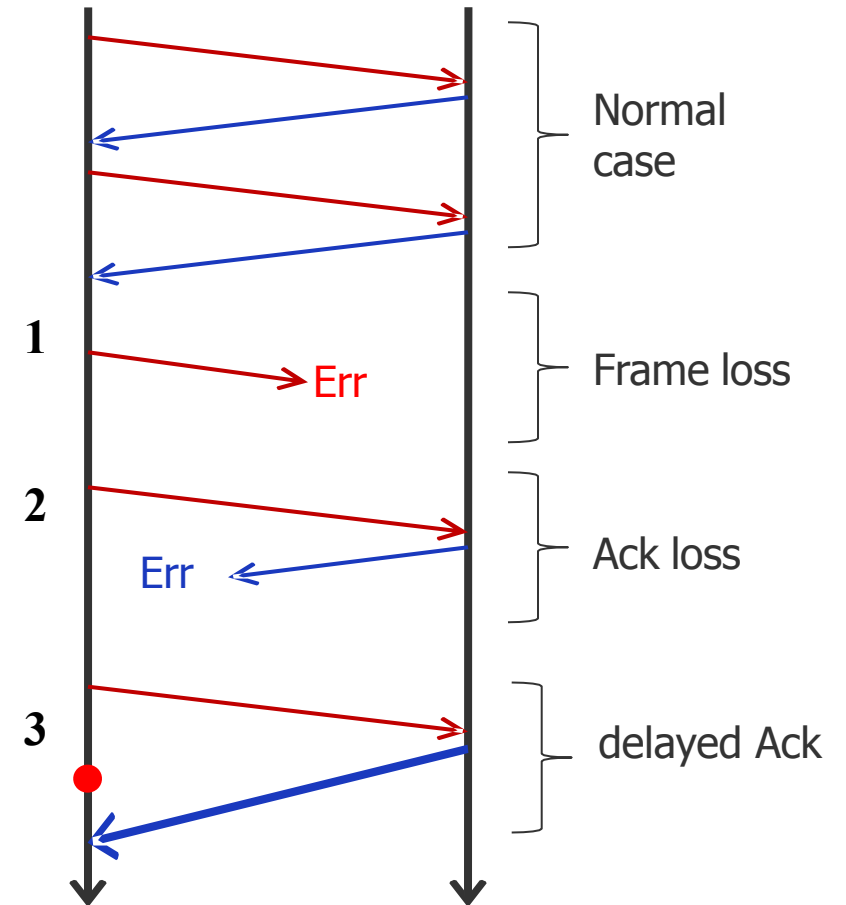
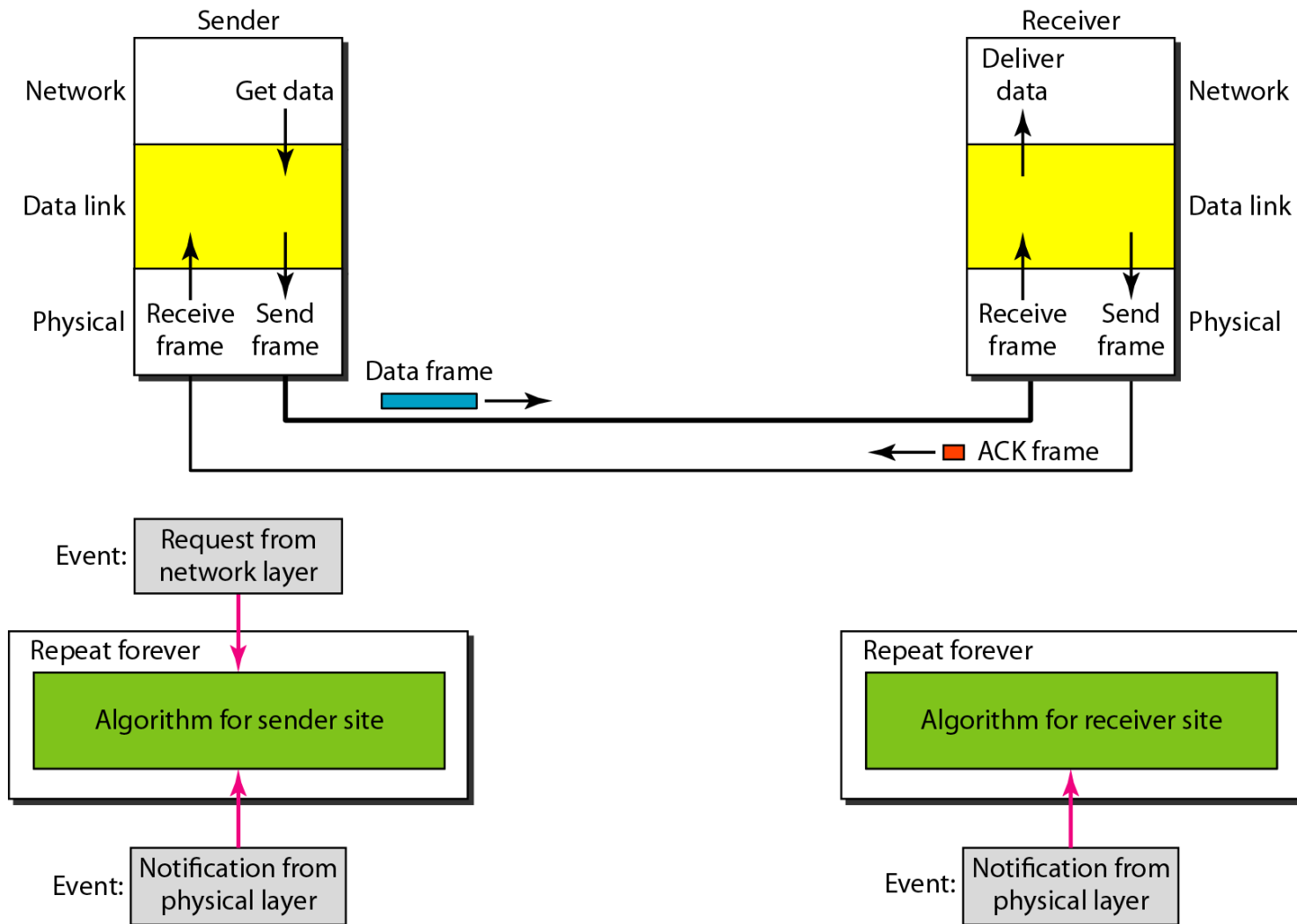


Figure 11.8 *Design of Stop-and-Wait Protocol*



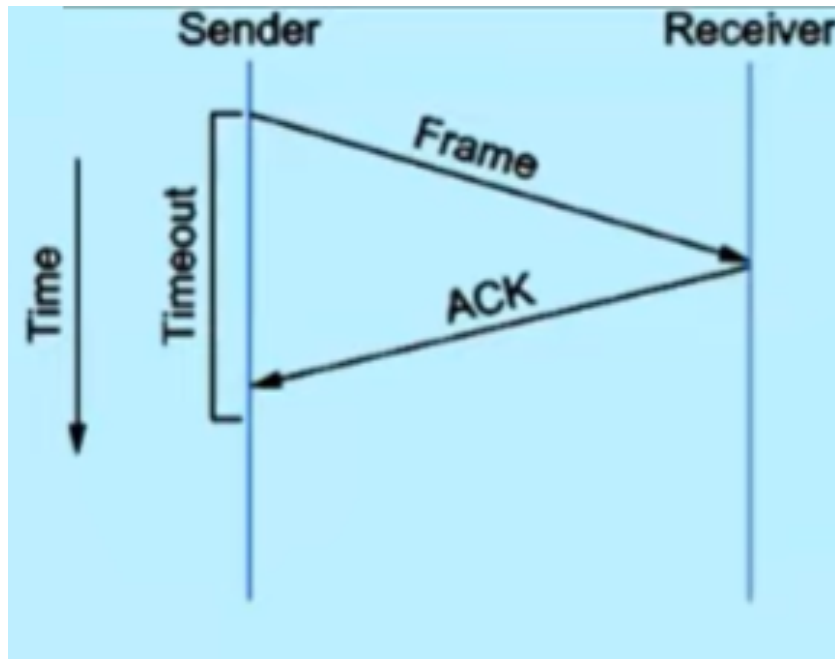
Algorithm *Sender-algorithm for Stop-and-Wait Protocol*

```
1 while(true)                                //Repeat forever
2   canSend = true                            //Allow the first frame to go
3   {
4     WaitForEvent();                          // Sleep until an event occurs
5     if(Event(RequestToSend) AND canSend)
6     {
7       GetData();
8       MakeFrame();
9       SendFrame();                          //Send the data frame
10      canSend = false;                      //Cannot send until ACK arrives
11    }
12    WaitForEvent();                          // Sleep until an event occurs
13    if(Event(ArrivalNotification) // An ACK has arrived
14    {
15      ReceiveFrame();                        //Receive the ACK frame
16      canSend = true;
17    }
18 }
```

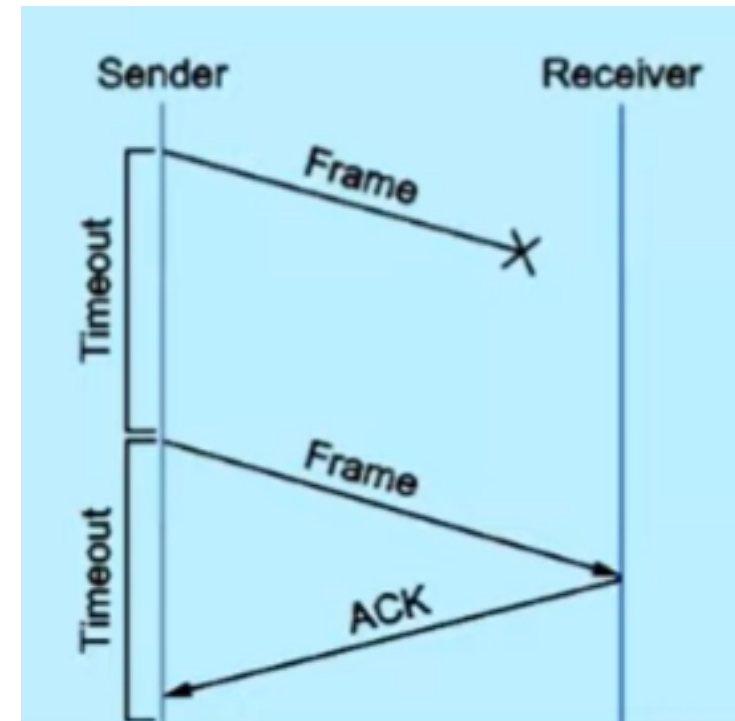

Data Link Layer Protocol - Stop and Wait ARQ Protocol

- ❖ In stop-and-wait protocol, after transmitting one frame, the sender waits for an acknowledgement before transmitting the next frame
- ❖ If the acknowledgement does not arrive after a certain period of time, the sender times out and retransmits the original frame
- ❖ Stop-and-wait ARQ protocol = stop-and-wait protocol + timeout timer + sequence number

Data Link Layer Protocol - Stop and Wait ARQ Protocol

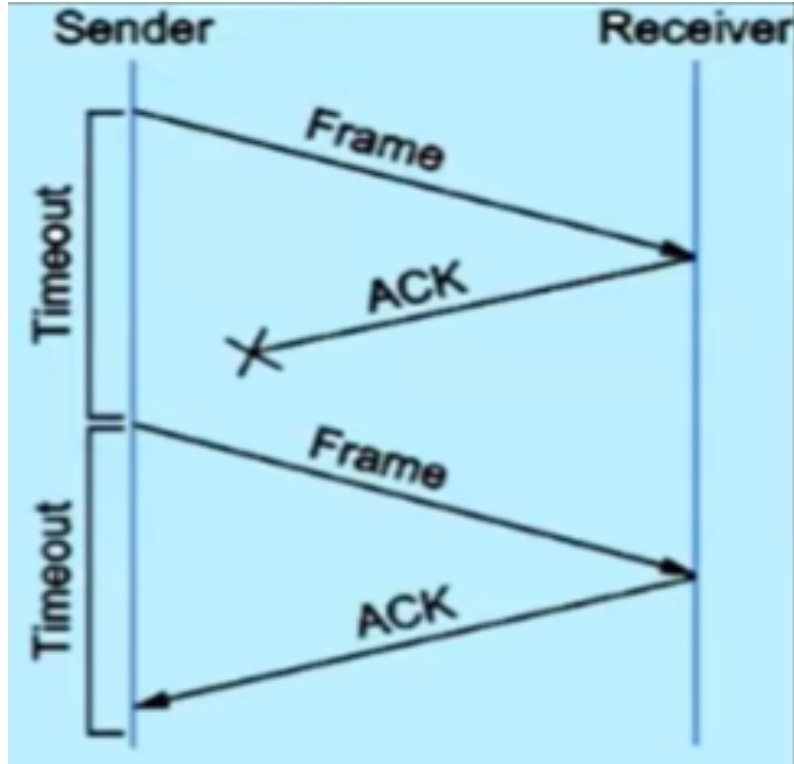


The Ack is received
before the timer expires

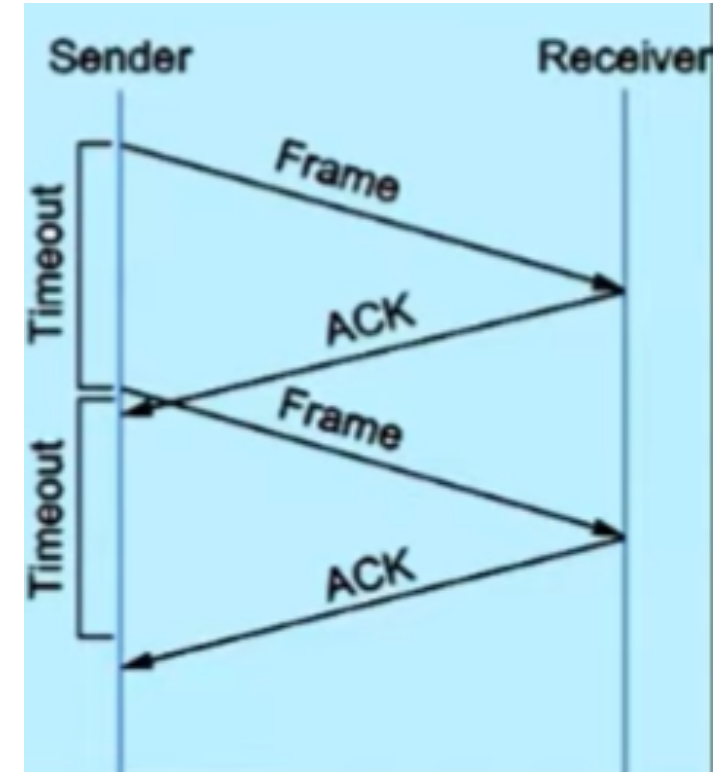


The original frame is lost

Data Link Layer Protocol - Stop and Wait ARQ Protocol



The Ack is lost

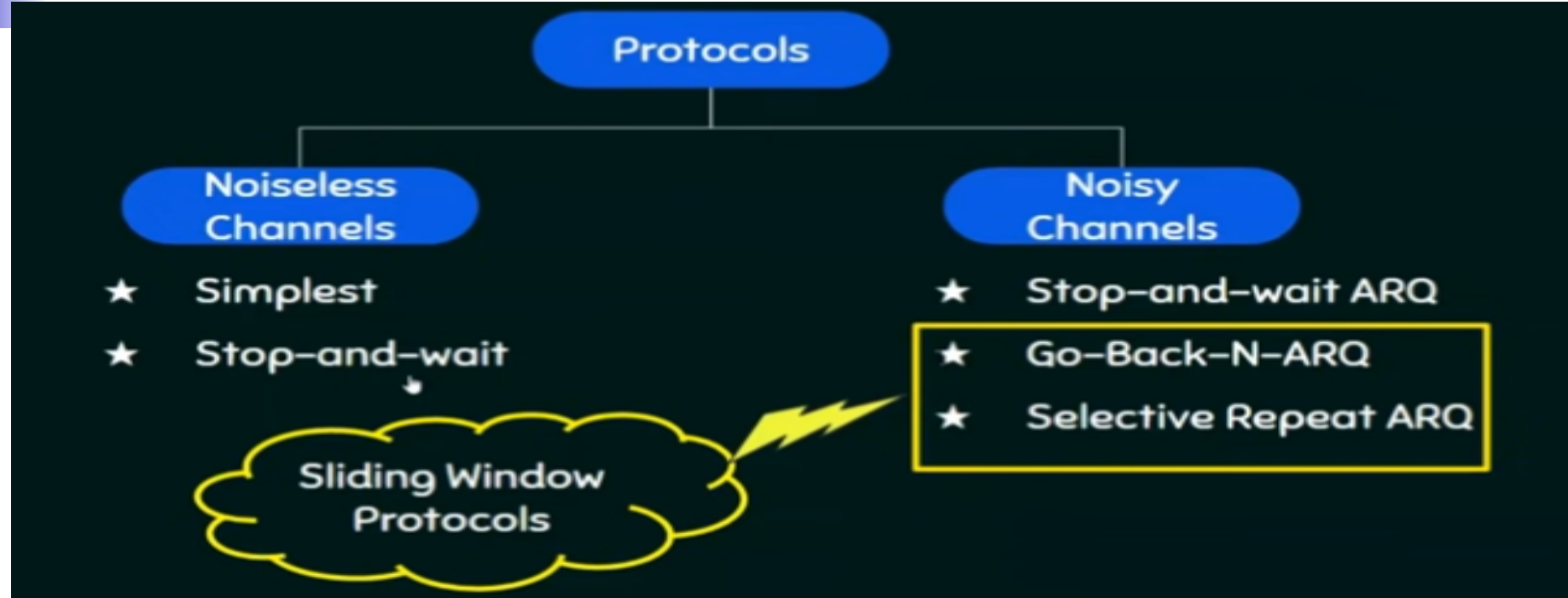


The timeout fires too soon

***Pipelining:** A task is begun before the previous task has ended*

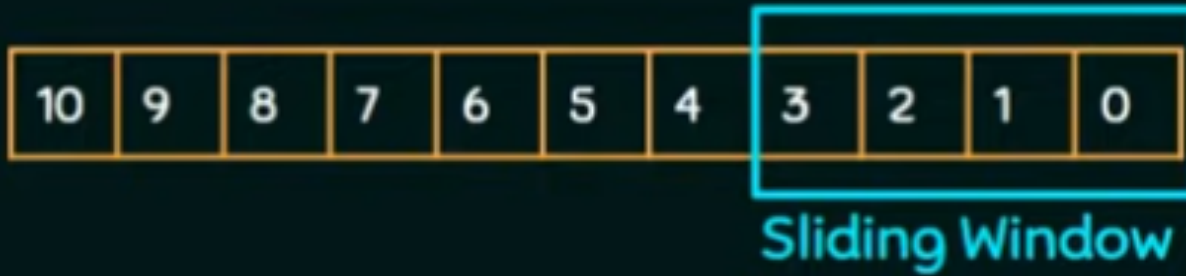
- ❖ *There is no pipelining in stop and wait ARQ because we need to wait for a frame to reach the destination and be acknowledged before the next frame can be sent*
- ❖ *Pipelining improves the efficiency of the transmission*

Sliding window protocol



- ❖ Send multiple frames at a time
- ❖ Number of frames to be sent is based on the **Window size**
- ❖ Each frame is numbered → sequence number

Sliding window protocol

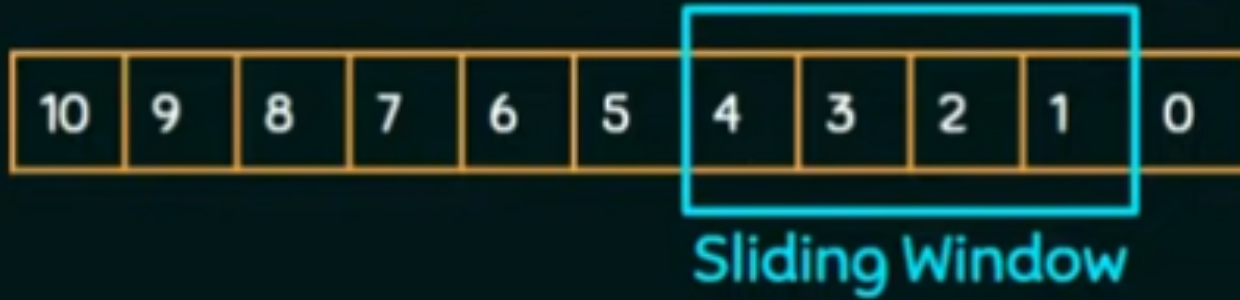


Window Size:

4

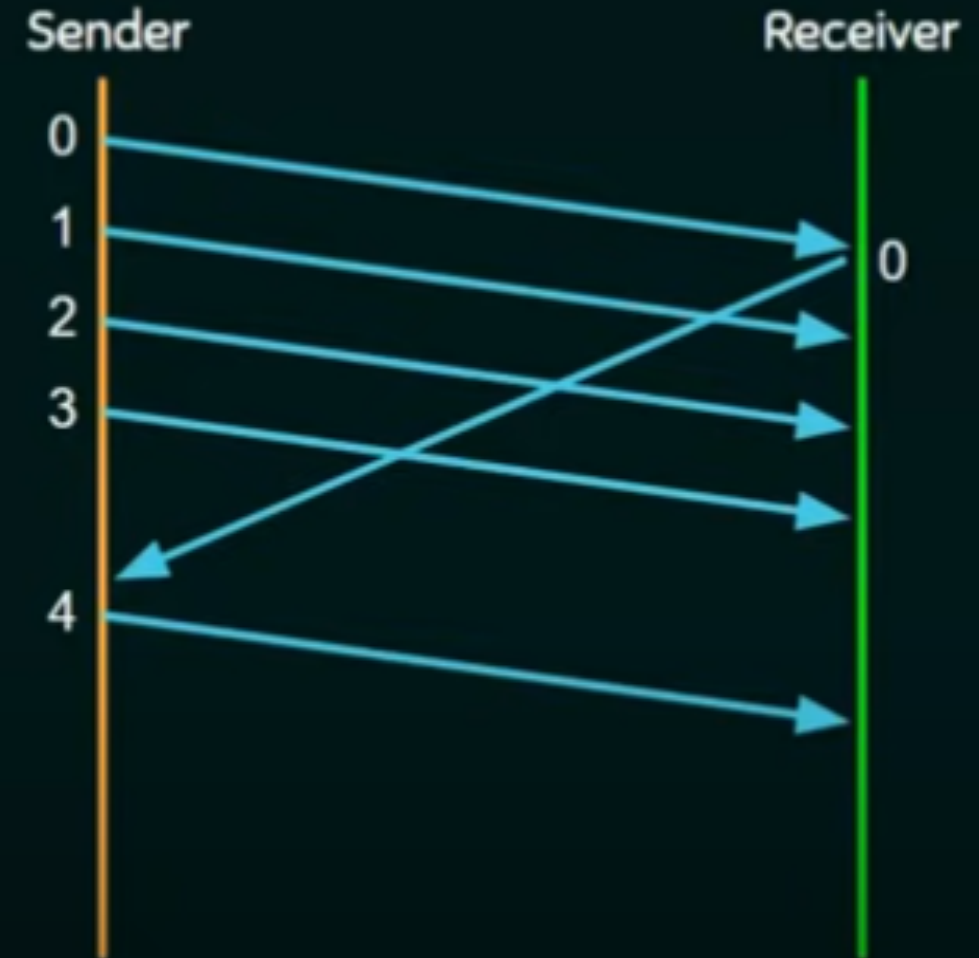


Sliding window protocol

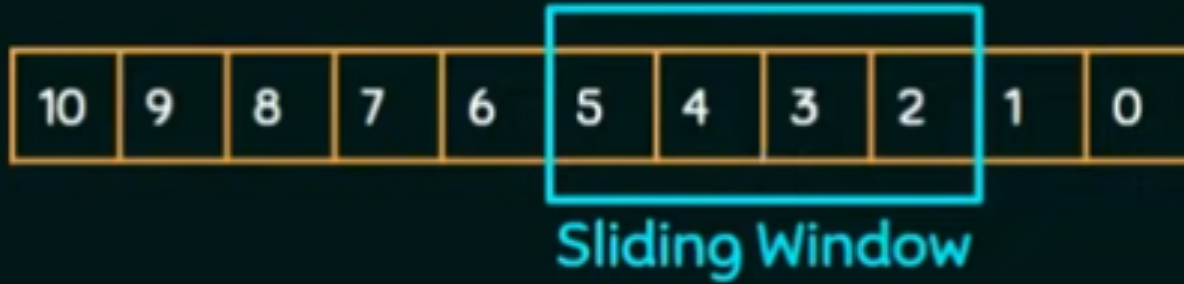


Window Size:

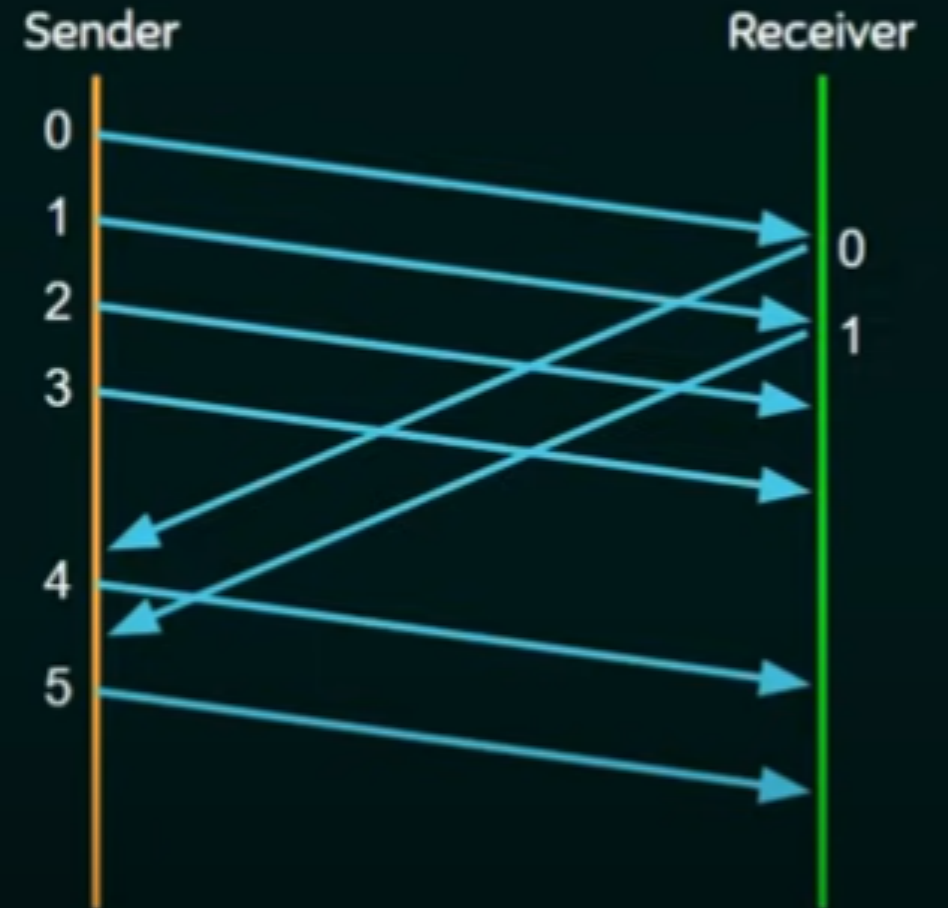
4



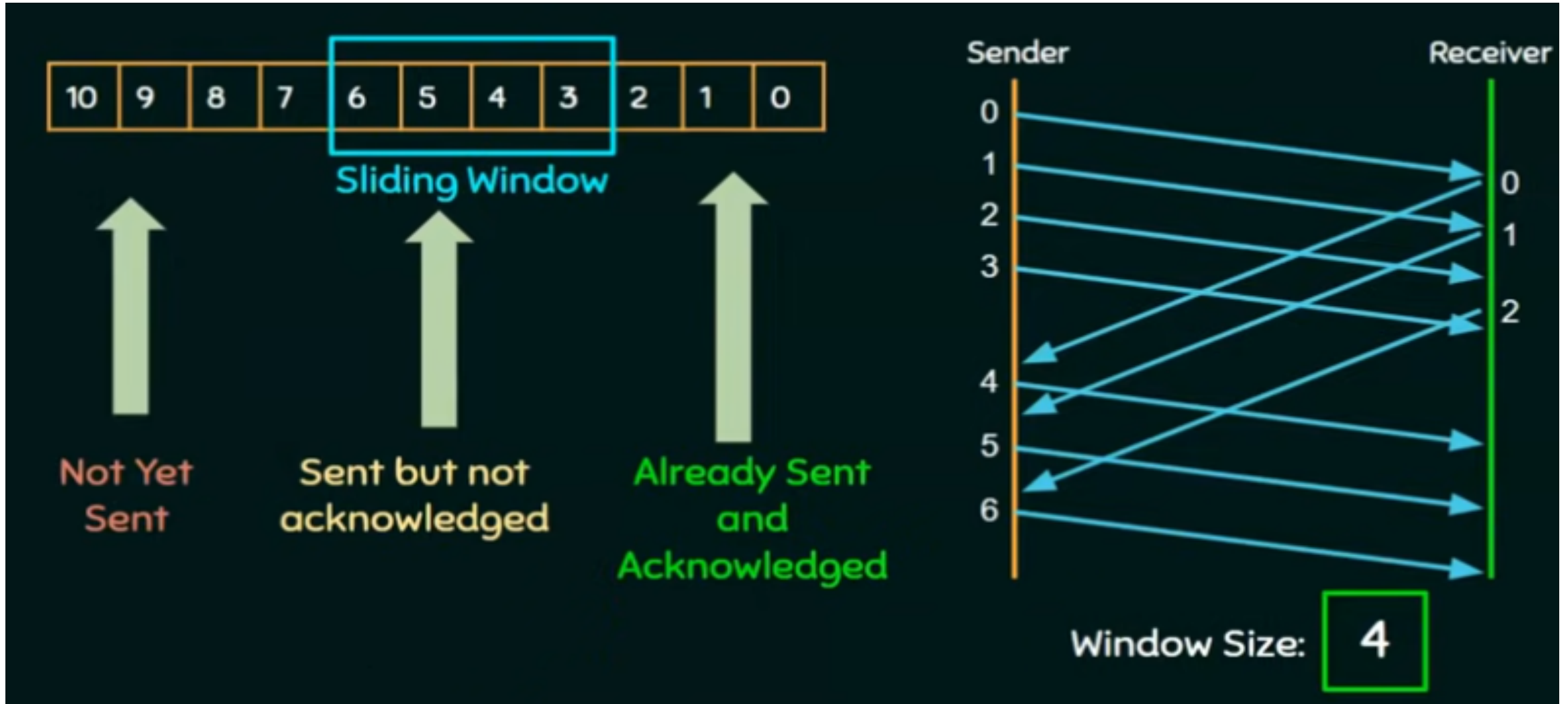
Sliding window protocol



Window Size: 4



Sliding window protocol



Go_Back_N ARQ

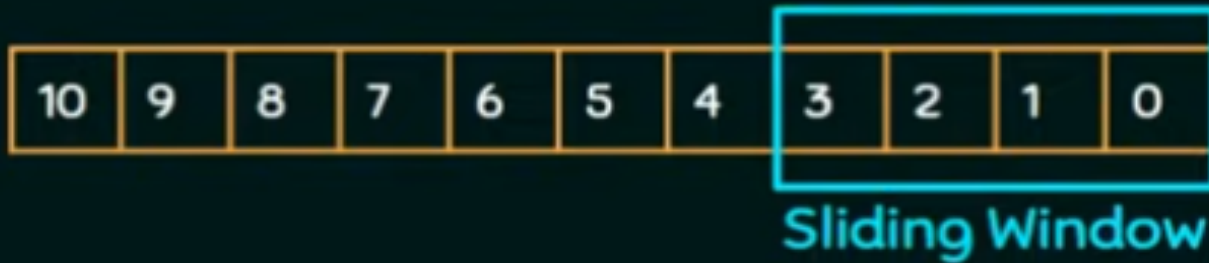
Go-Back-N ARQ

'N' is the sender window size.

- ❖ It uses the concept of protocol pipelining (i.e.) the sender can send multiple frames before receiving the acknowledgement for the first frame
- ❖ There are finite number of frames and frames are numbered in a sequential manner
- ❖ The number of frames that can be sent depends on the window size of the sender
- ❖ If the acknowledgment of the frame is not received within an agreed upon time period, all the frames in the current window are transmitted.

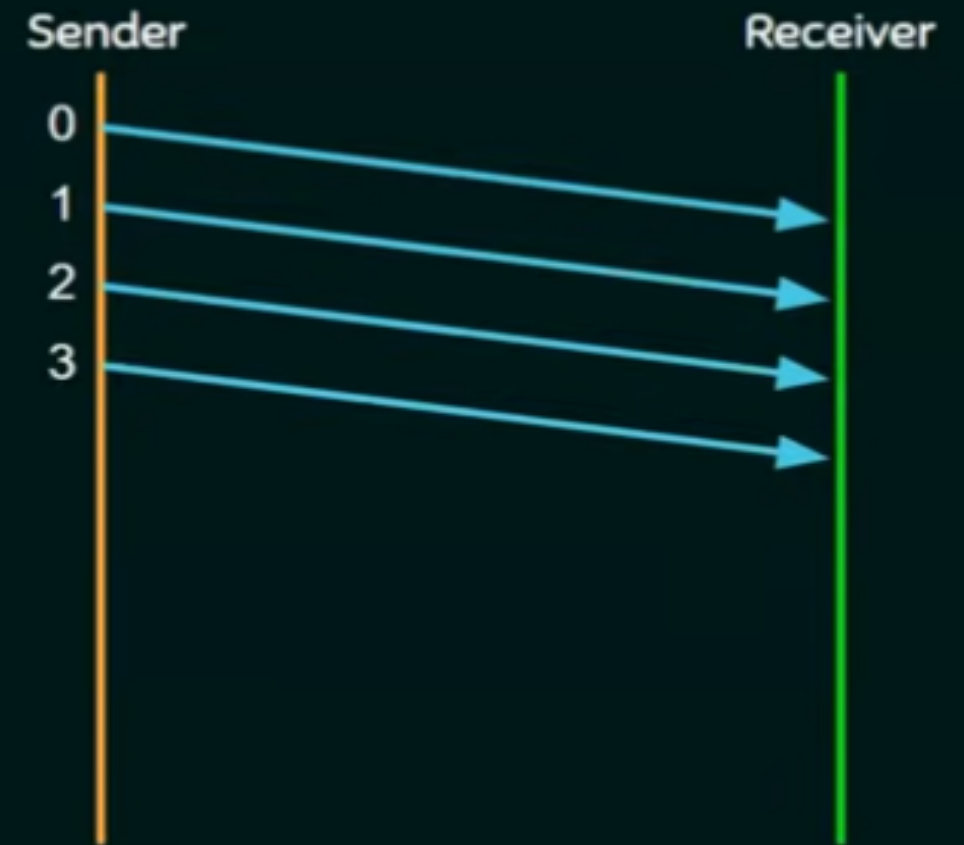
Go_Back_N ARQ

Working



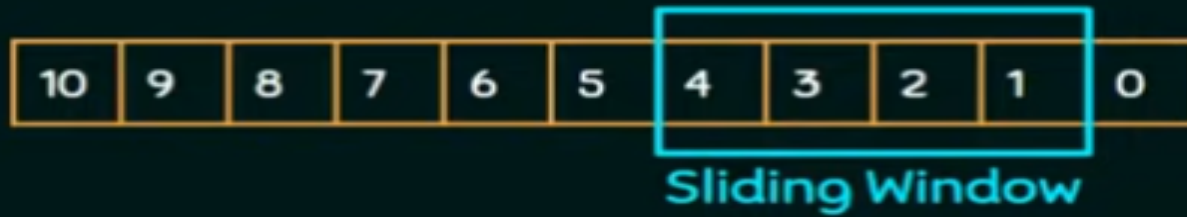
Window Size:

4

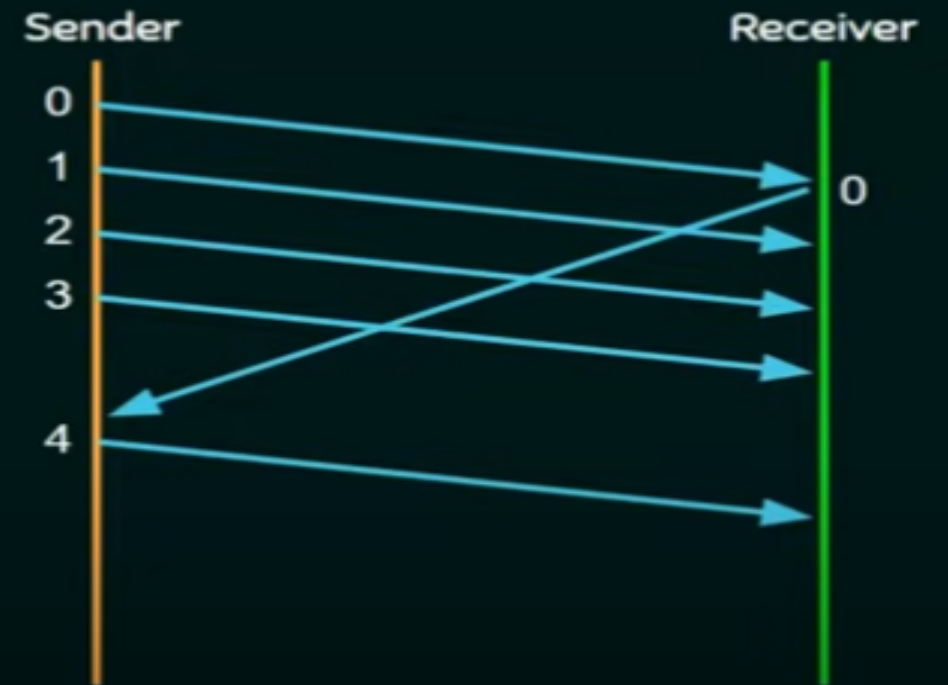


Go_Back_N ARQ

Working

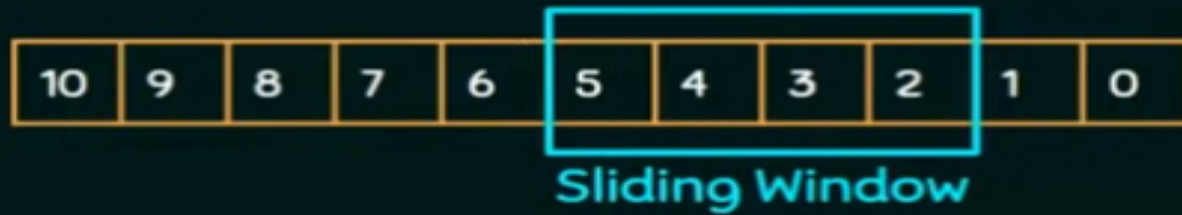


Window Size: 4



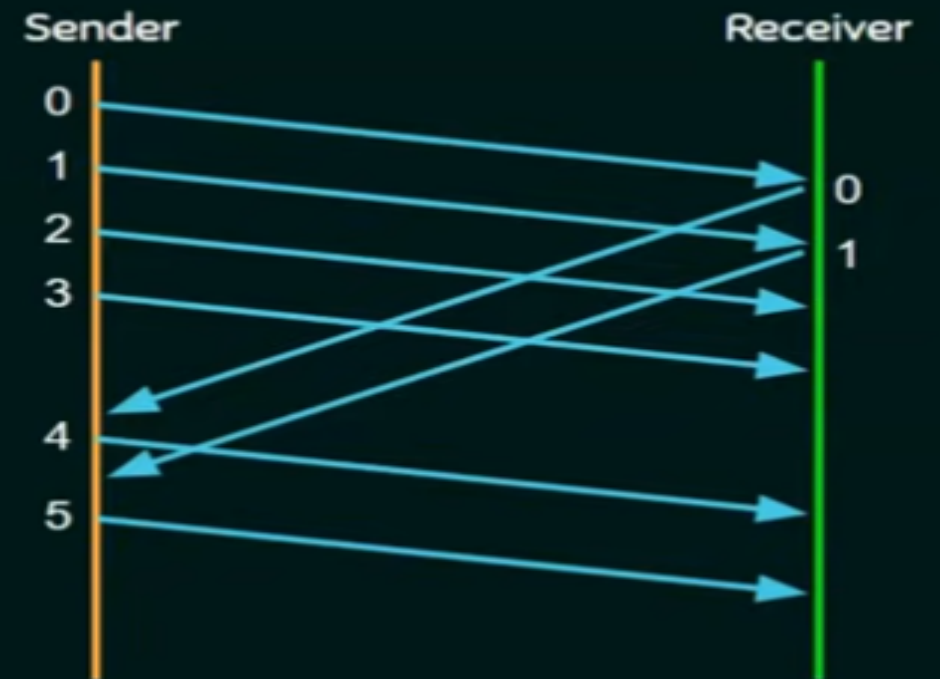
Go_Back_N ARQ

Working



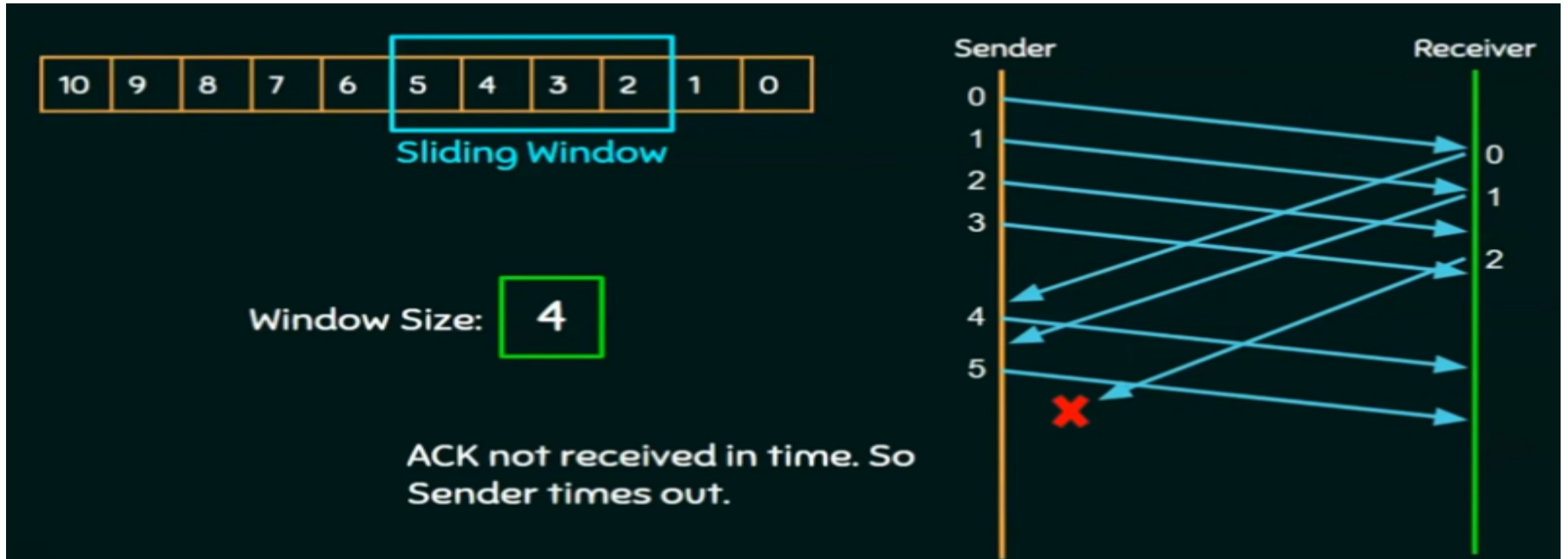
Window Size:

4



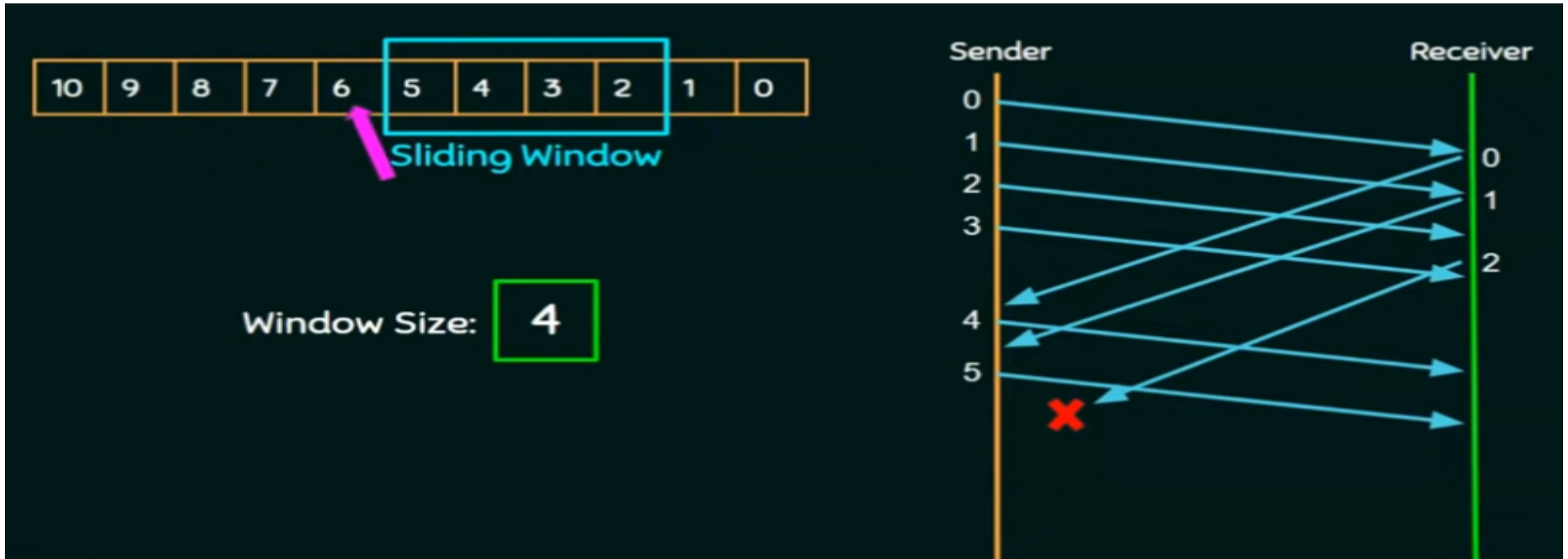
Go_Back_N ARQ

Working



Go_Back_N ARQ

Working



Go_Back_N ARQ

Working



Figure *Design of Go-Back-N ARQ*

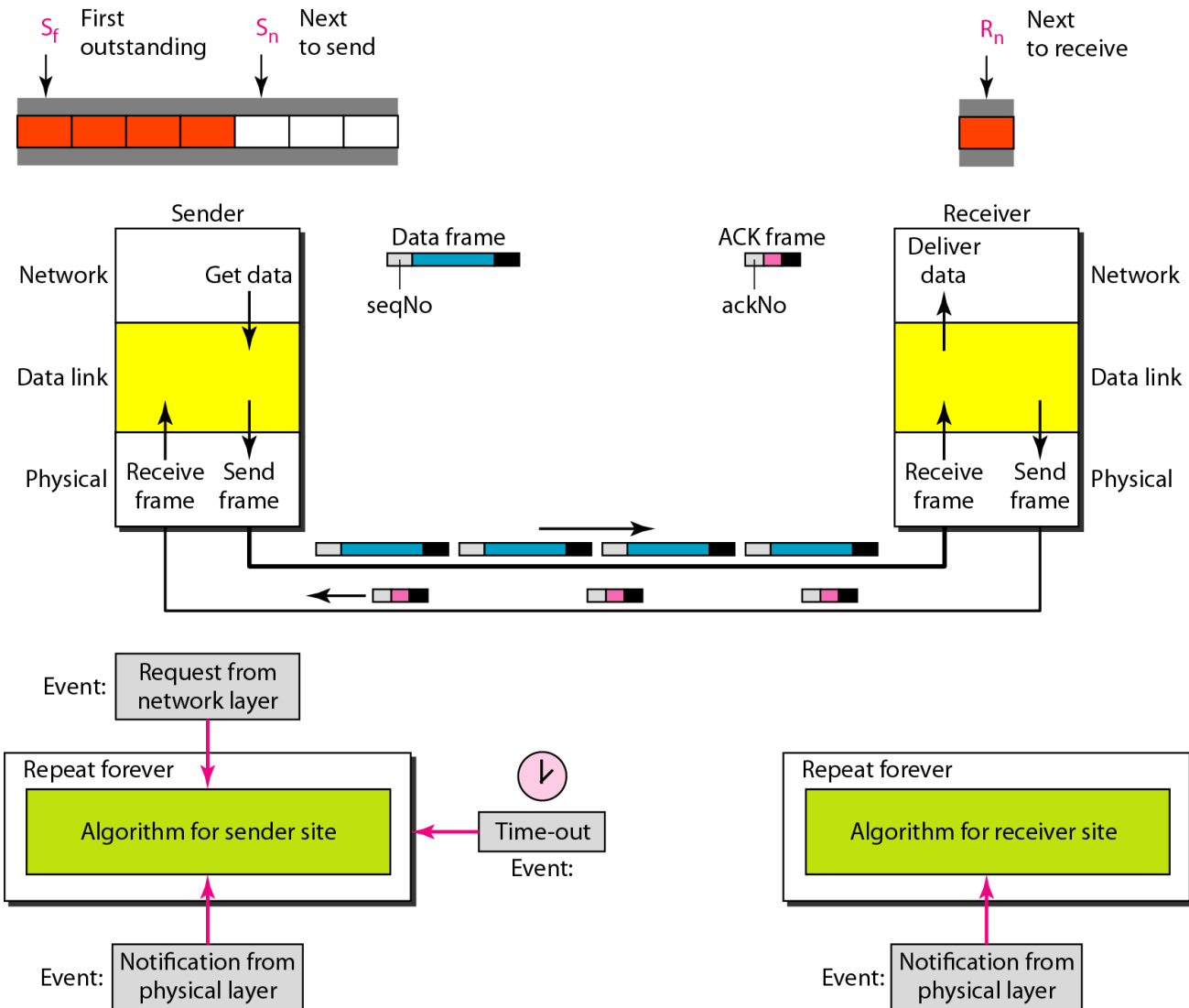
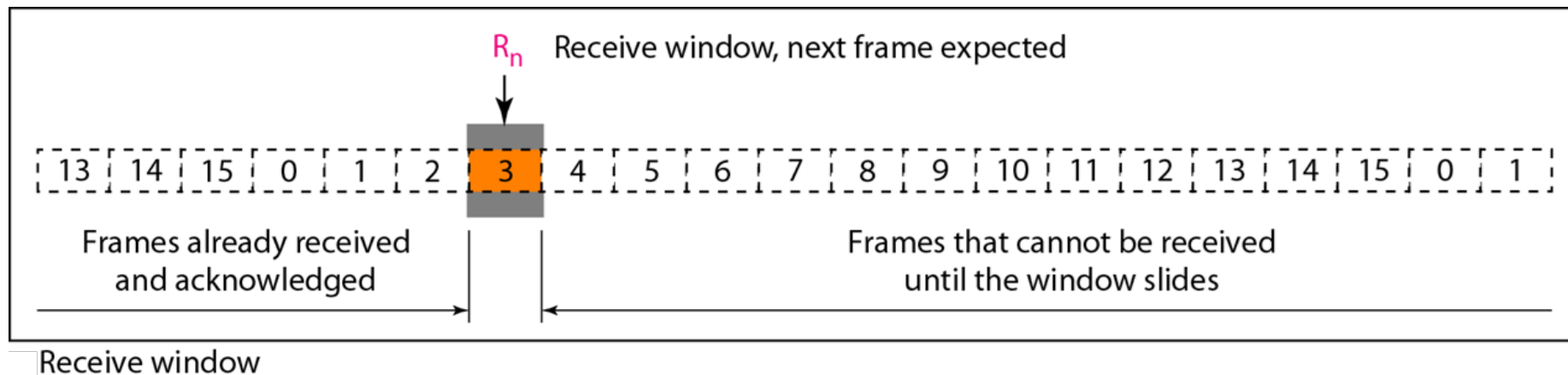
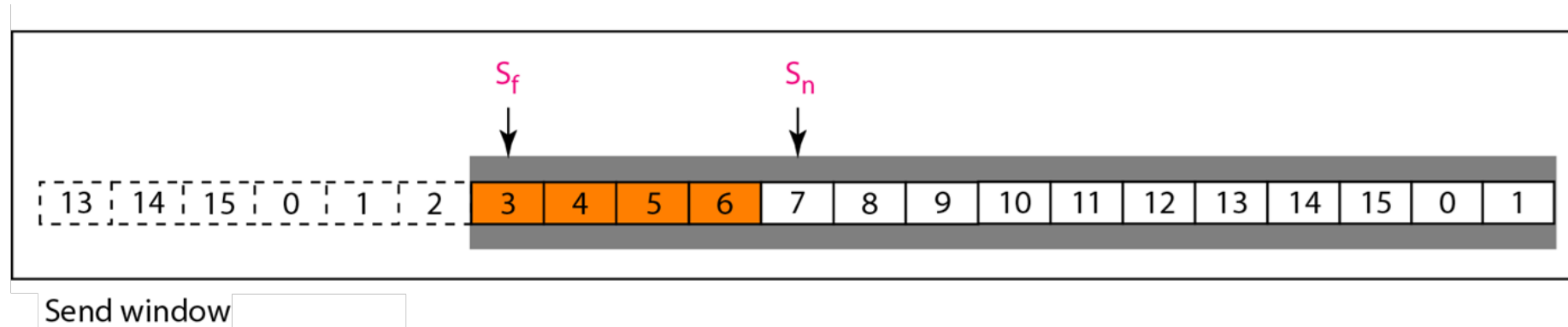
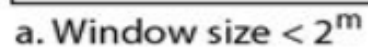


Figure *Send and Receive window for Go-Back-N ARQ*



Sender window size $< (2^m)$, Receiver size=1;
 $M=2$; Sender window size $< 4=3$, Receiver size=1



Sender window size $= (2^m)$, Receiver size $= 1$;
 $M=2$; Sender window size < 4 , Receiver size $= 1$;





In the **Go-Back-N Protocol**, the sequence numbers are modulo 2^m , where m is the size of the sequence number field in bits.

The send window is an abstract concept defining an imaginary box of size $2^m - 1$ with three variables: S_f , S_n , and S_{size} .

The send window can slide one slot when a valid acknowledgment arrives.

In Go-Back-N ARQ, the size of the send window must be less than 2^m ; the size of the receiver window is always 1.

Algorithm *Go-Back-N sender algorithm*

```
1   $S_w = 2^m - 1;$ 
2   $S_f = 0;$ 
3   $S_n = 0;$ 
4
5  while (true)                                //Repeat forever
6  {
7      WaitForEvent();
8      if(Event(RequestToSend))                //A packet to send
9      {
10         if( $S_n - S_f \geq S_w$ )                 //If window is full
11             Sleep();
12         GetData();
13         MakeFrame( $S_n$ );
14         StoreFrame( $S_n$ );
15         SendFrame( $S_n$ );
16          $S_n = S_n + 1;$ 
17         if(timer not running)
18             StartTimer();
19     }
20
```

(continued)

Algorithm *Go-Back-N sender algorithm*

(continued)

```
21  if (Event (ArrivalNotification))  //ACK arrives
22  {
23      Receive (ACK);
24      if (corrupted (ACK))
25          Sleep();
26      if ((ackNo > Sf) && (ackNo ≤ Sn))  //If a valid ACK
27      While (Sf ≤ ackNo)
28      {
29          PurgeFrame (Sf);
30          Sf = Sf + 1;
31      }
32      StopTimer();
33  }
34
35  if (Event (TimeOut))  //The timer expires
36  {
37      StartTimer();
38      Temp = Sf;
39      while (Temp < Sn);
40      {
41          SendFrame (Sf);
42          Sf = Sf + 1;
43      }
44  }
45 }
```

Algorithm *Go-Back-N receiver algorithm*

```
1  Rn = 0;
2
3  while (true)                                //Repeat forever
4  {
5      WaitForEvent();
6
7      if(Event(ArrivalNotification))           //Data frame arrives
8      {
9          Receive(Frame);
10         if(corrupted(Frame))
11             Sleep();
12         if(seqNo == Rn)                       //If expected frame
13         {
14             DeliverData();                     //Deliver data
15             Rn = Rn + 1;                       //Slide window
16             SendACK(Rn);
17         }
18     }
19 }
```



Selective Repeat ARQ

*Go-Back-N ARQ is inefficient of a **noisy** link.*

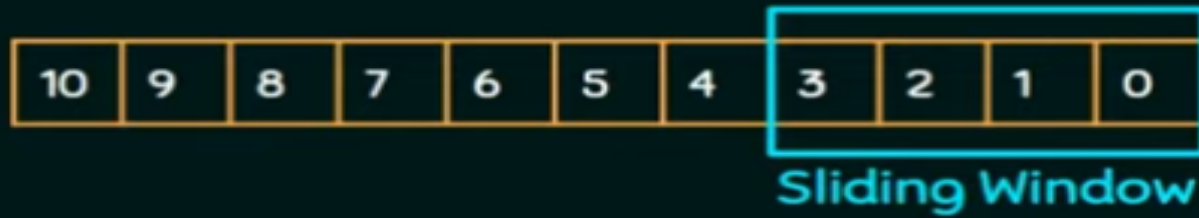
- *In a noisy link frames have higher probability of damage , which means the resending of multiple frames.*
- *this resending consumes the bandwidth and slow down the transmission .*

Solution:

- *Selective Repeat ARQ protocol : resent only the damage frame*
- *It defines a negative Acknowledgement (NAK) that report the sequence number of a damaged frame before the timer expires*
- *Correct frames are received and buffered*
- *It is more efficient for noisy link, but the processing at the receiver is more complex*

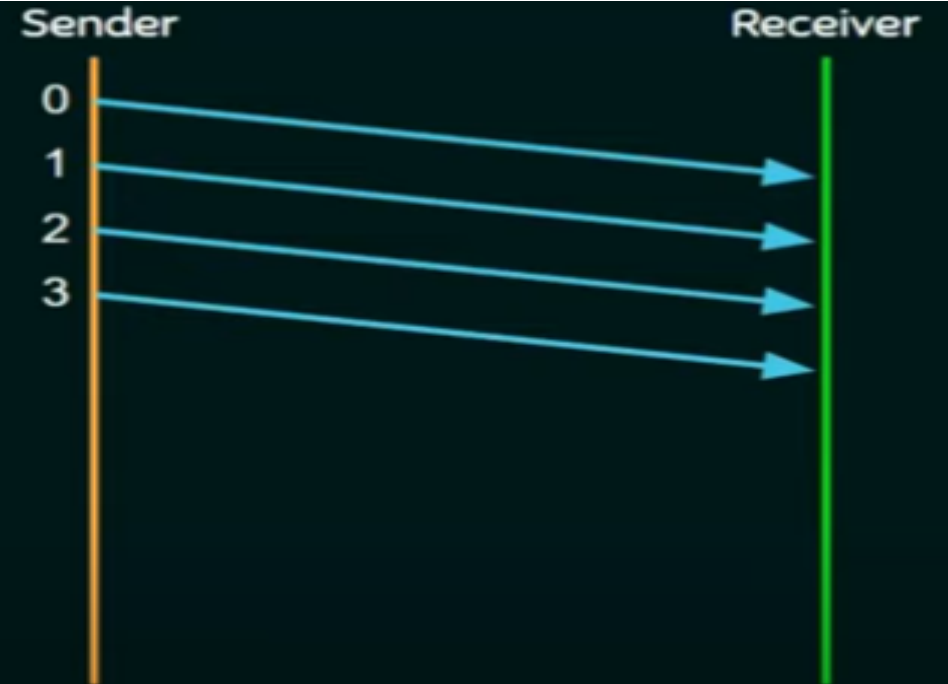
Selective Repeat ARQ

Working



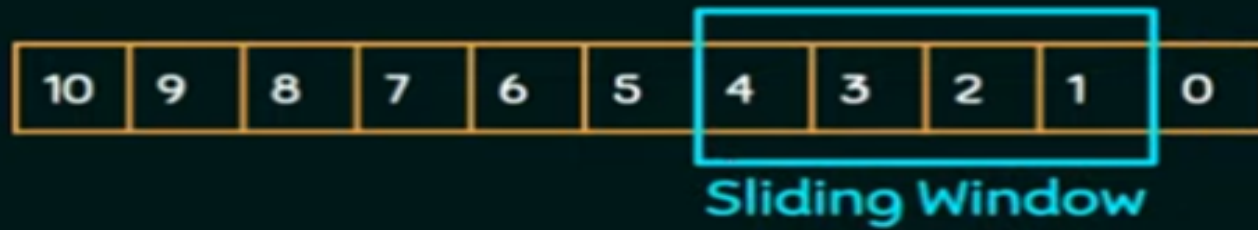
Window Size:

4



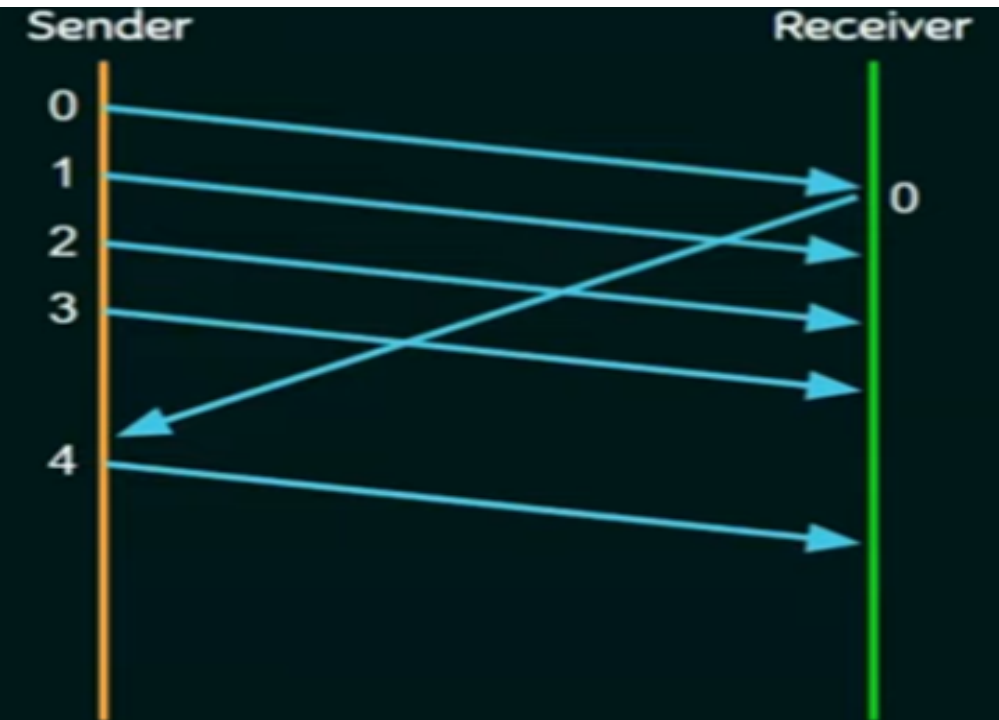
Selective Repeat ARQ

Working



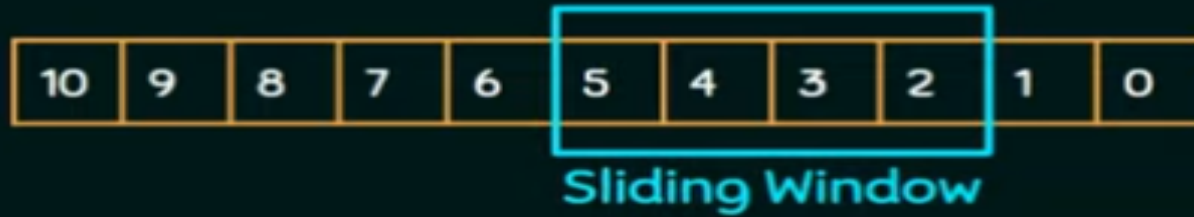
Window Size:

4



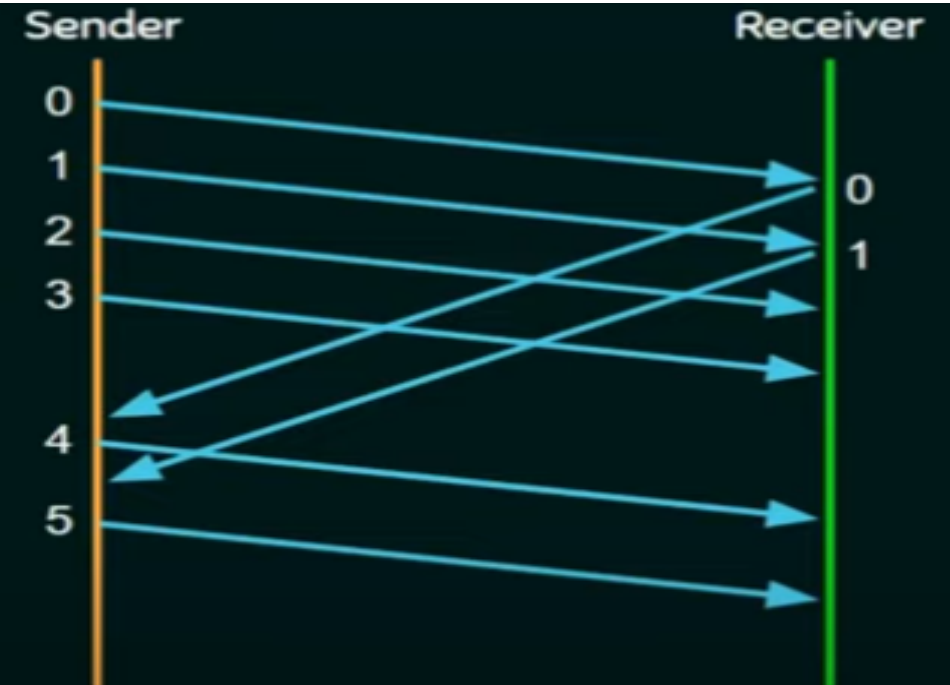
Selective Repeat ARQ

Working



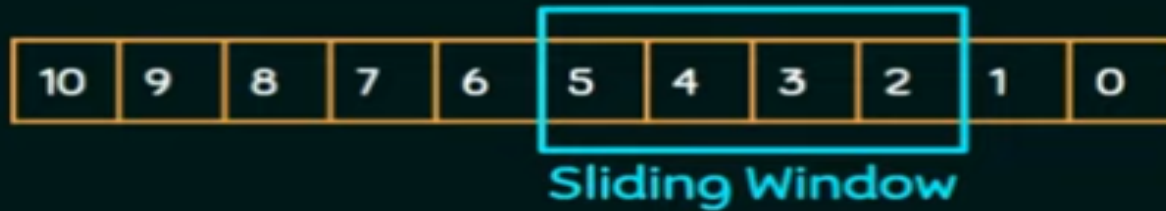
Window Size:

4



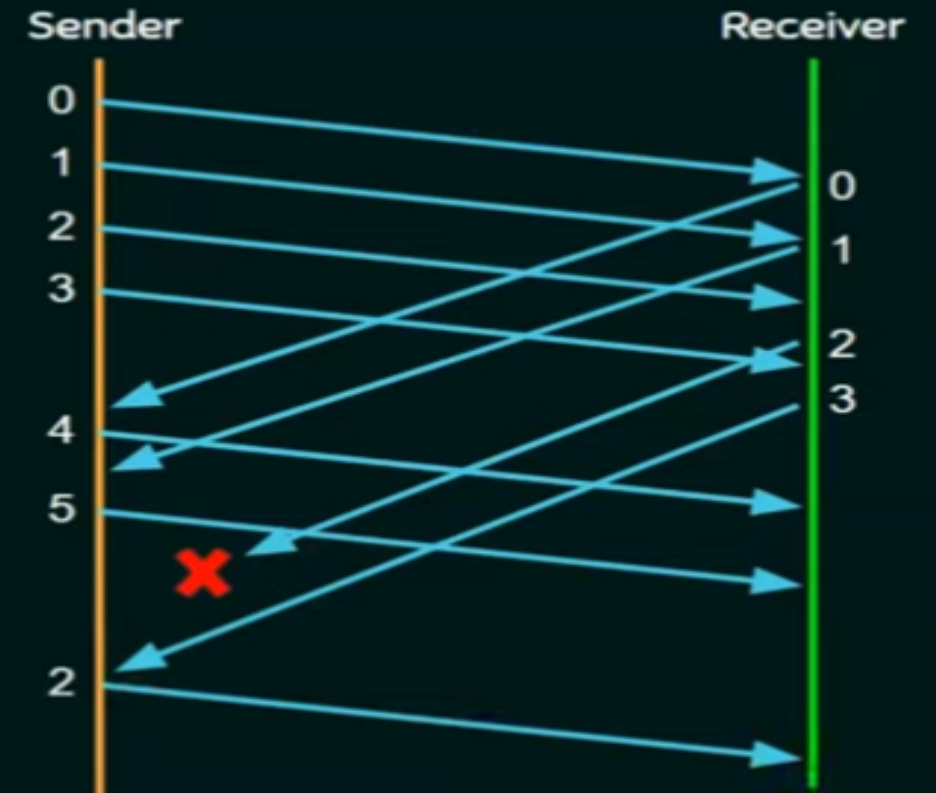
Selective Repeat ARQ

Working



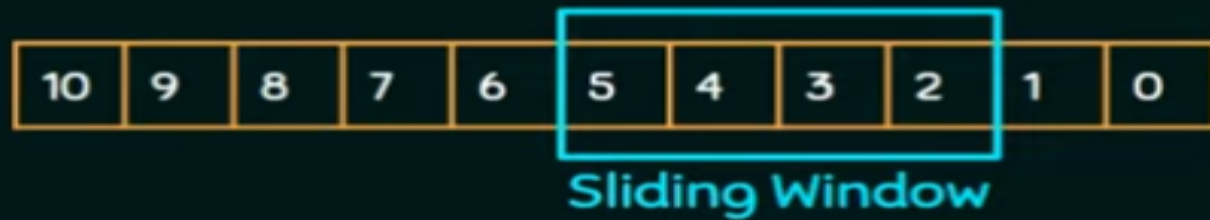
Window Size:

4



Selective Repeat ARQ

Working



Window Size:

4

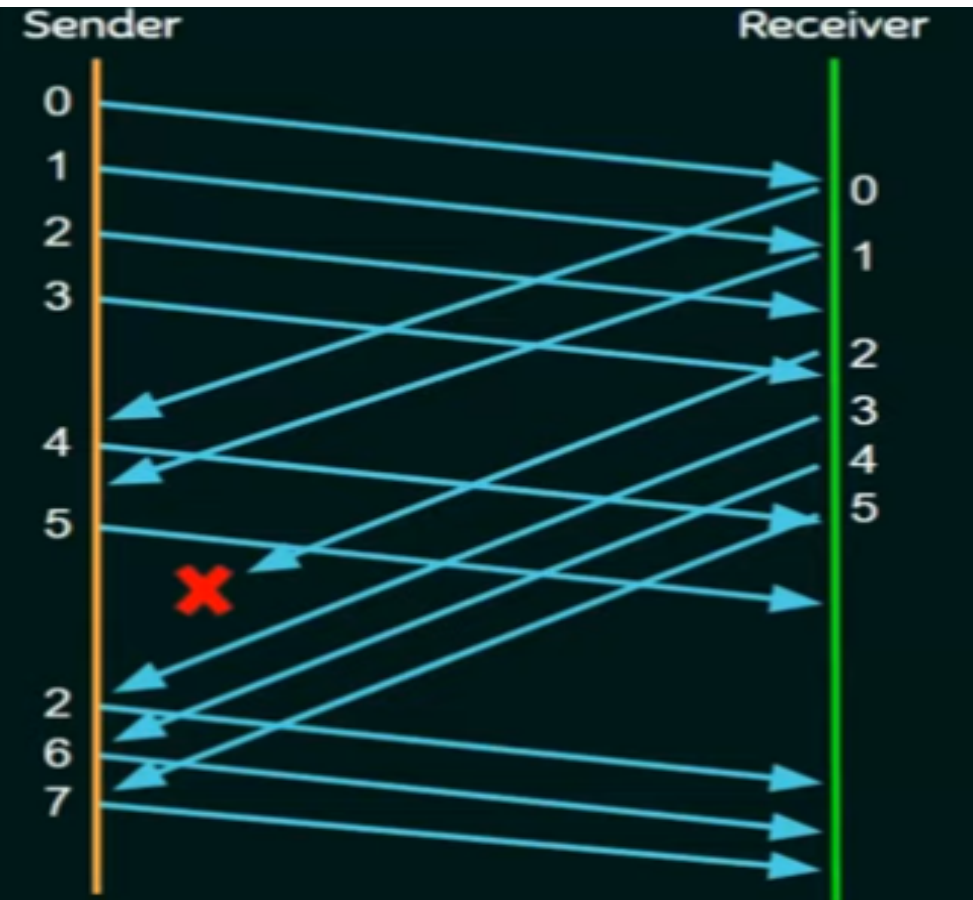


Figure *Send window for Selective Repeat ARQ*

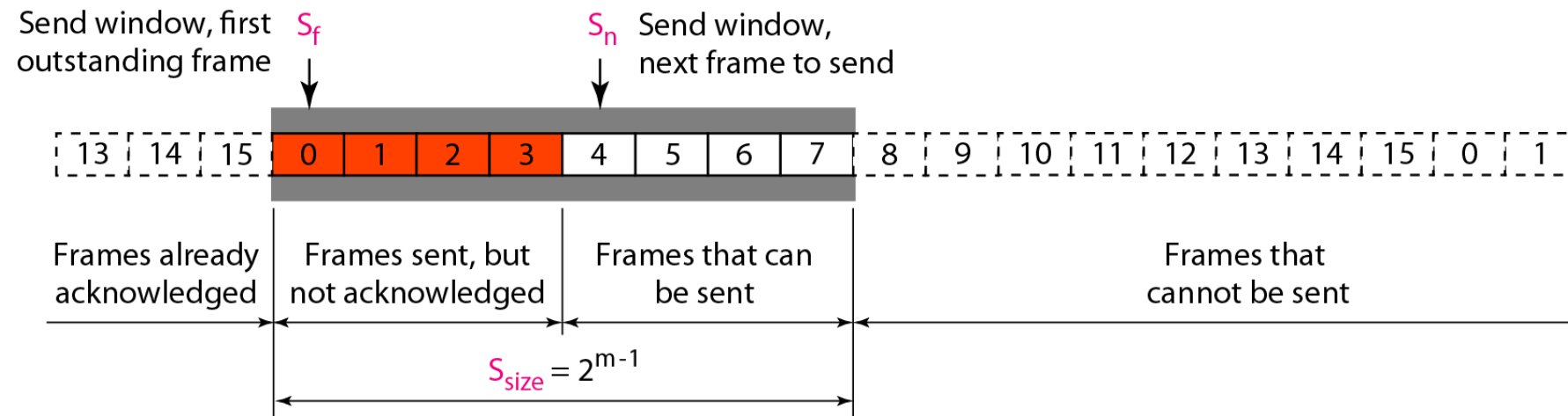


Figure *Receive window for Selective Repeat ARQ*

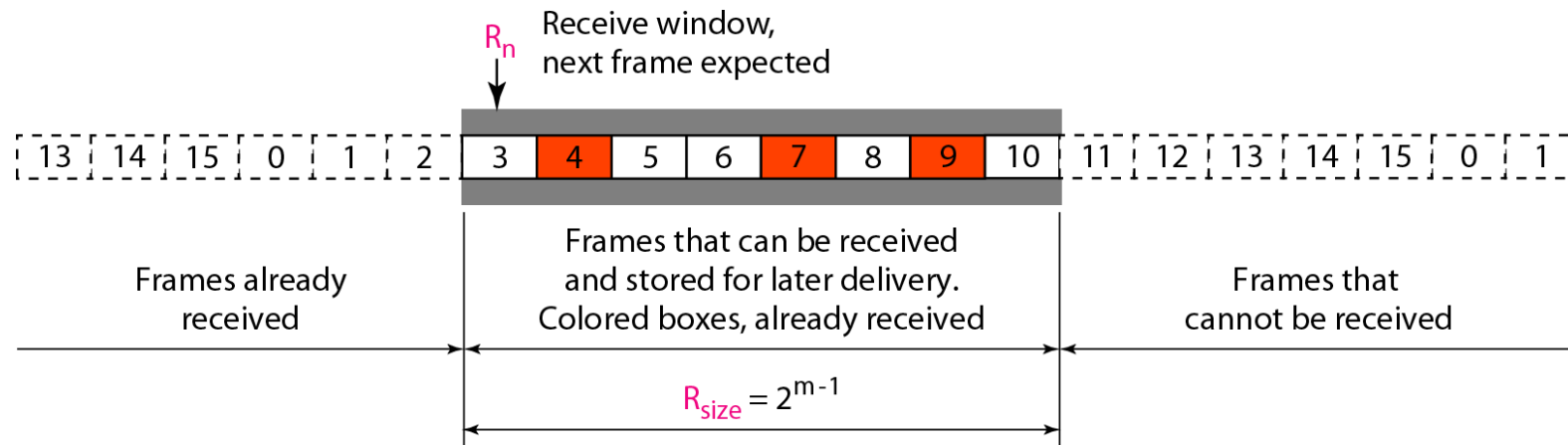
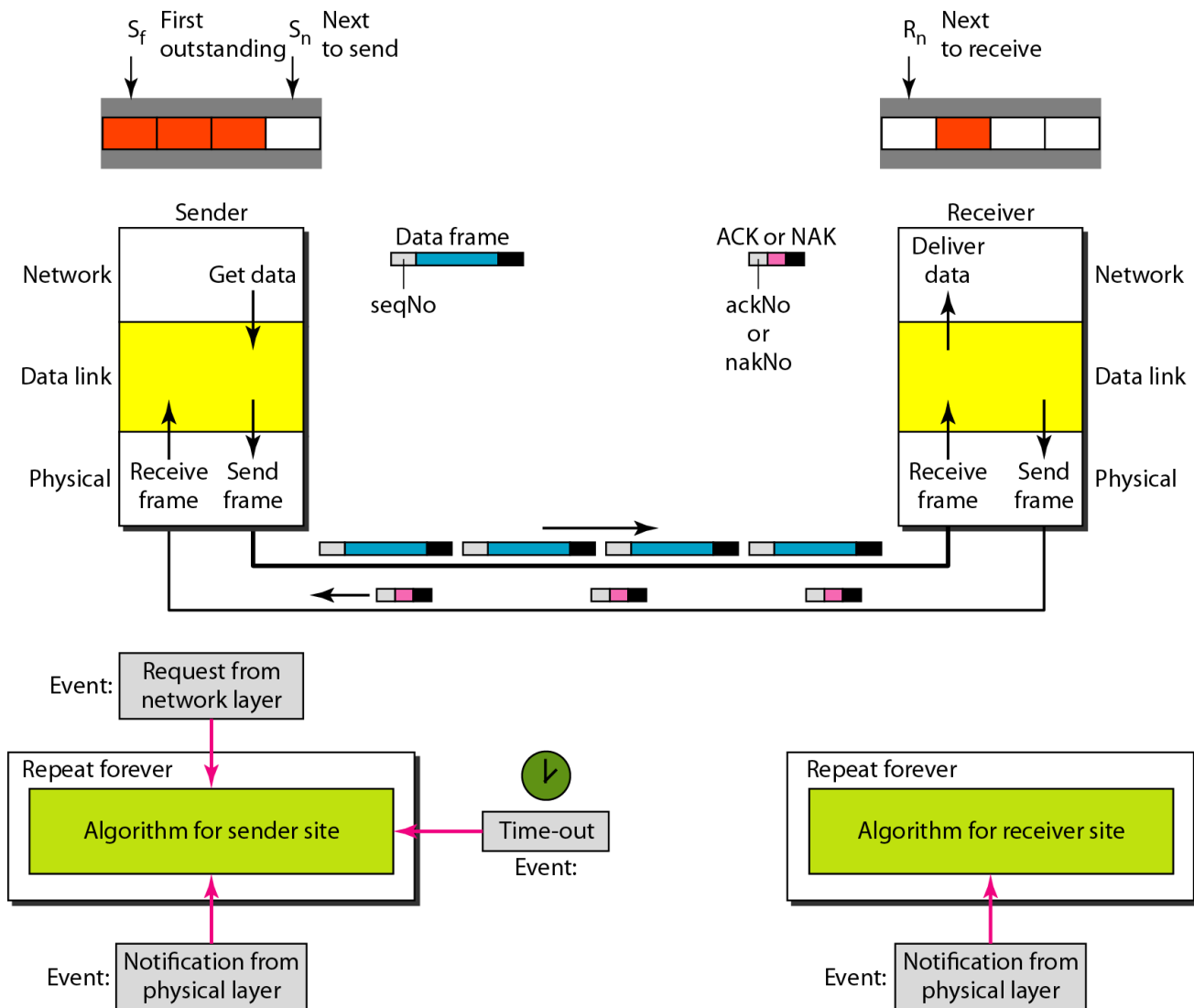


Figure *Design of Selective Repeat ARQ*



Algorithm *Sender-site Selective Repeat algorithm*

```
1   $S_w = 2^{m-1}$  ;
2   $S_f = 0$  ;
3   $S_n = 0$  ;
4
5  while (true)                                //Repeat forever
6  {
7      WaitForEvent() ;
8      if(Event(RequestToSend))                //There is a packet to send
9      {
10         if( $S_n - S_f \geq S_w$ )                  //If window is full
11             Sleep() ;
12         GetData() ;
13         MakeFrame( $S_n$ ) ;
14         StoreFrame( $S_n$ ) ;
15         SendFrame( $S_n$ ) ;
16          $S_n = S_n + 1$  ;
17         StartTimer( $S_n$ ) ;
18     }
19
```

(continued)

Algorithm *Sender-site Selective Repeat algorithm*

(continued)

```
20  if(Event(ArrivalNotification)) //ACK arrives
21  {
22      Receive(frame); //Receive ACK or NAK
23      if(corrupted(frame))
24          Sleep();
25      if (FrameType == NAK)
26          if (nakNo between  $S_f$  and  $S_n$ )
27          {
28              resend(nakNo);
29              StartTimer(nakNo);
30          }
31      if (FrameType == ACK)
32          if (ackNo between  $S_f$  and  $S_n$ )
33          {
34              while( $s_f < \text{ackNo}$ )
35              {
36                  Purge( $s_f$ );
37                  StopTimer( $s_f$ );
38                   $S_f = S_f + 1$ ;
39              }
40          }
41  }
```

(continued)

Algorithm *Sender-site Selective Repeat algorithm*

(continued)

```
42  
43   if (Event (TimeOut (t)))           //The timer expires  
44   {  
45       StartTimer(t);  
46       SendFrame(t);  
47   }  
48 }
```

Algorithm *Receiver-site Selective Repeat algorithm*

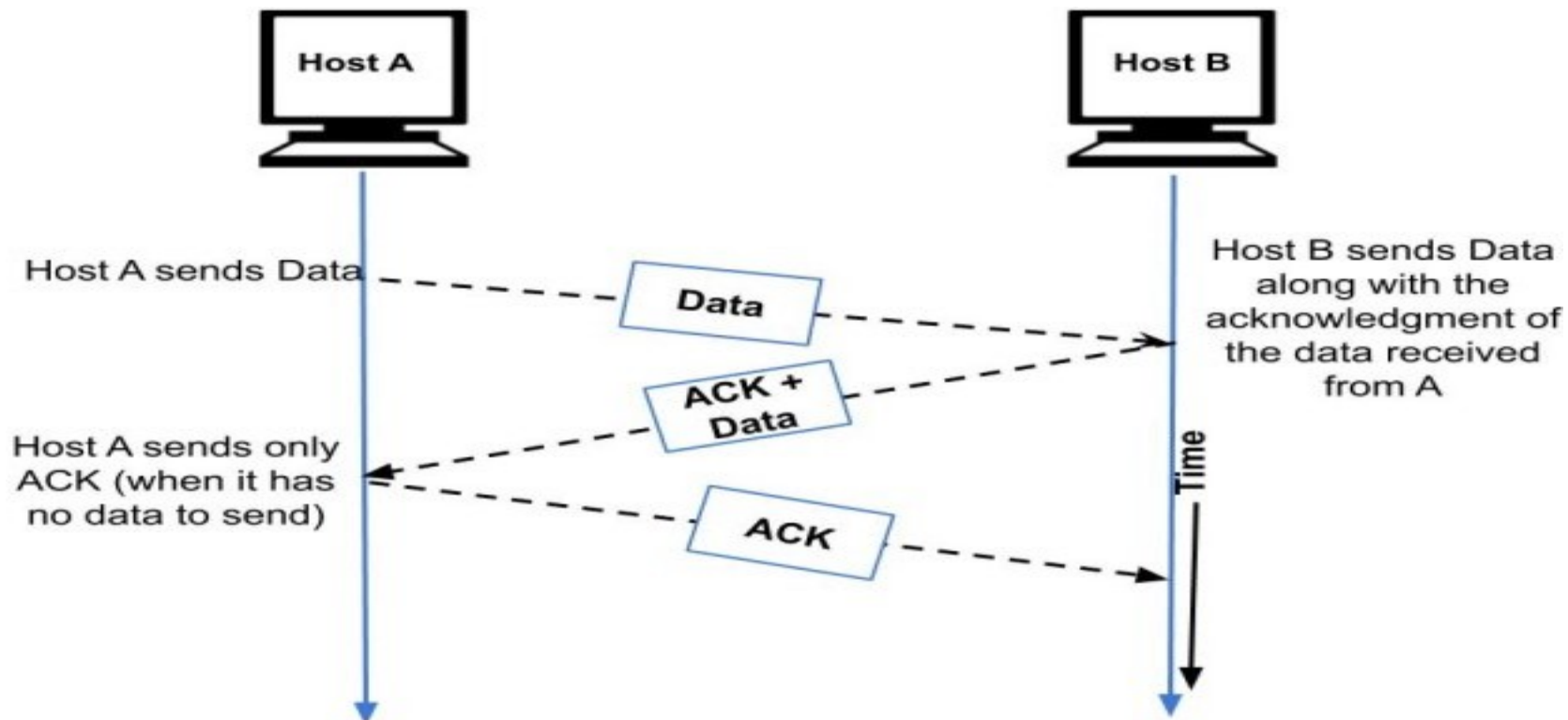
```
1  Rn = 0;
2  NakSent = false;
3  AckNeeded = false;
4  Repeat(for all slots)
5      Marked(slot) = false;
6
7  while (true)                                //Repeat forever
8  {
9      WaitForEvent();
10
11     if(Event(ArrivalNotification))           /Data frame arrives
12     {
13         Receive(Frame);
14         if(corrupted(Frame)) && (NOT NakSent)
15         {
16             SendNAK(Rn);
17             NakSent = true;
18             Sleep();
19         }
20         if(seqNo <> Rn) && (NOT NakSent)
21         {
22             SendNAK(Rn);
```

Algorithm *Receiver-site Selective Repeat algorithm*

```
23     NakSent = true;
24     if ((seqNo in window)&&(!Marked(seqNo)))
25     {
26         StoreFrame(seqNo)
27         Marked(seqNo)= true;
28         while(Marked(Rn))
29         {
30             DeliverData(Rn);
31             Purge(Rn);
32             Rn = Rn + 1;
33             AckNeeded = true;
34         }
35         if(AckNeeded);
36         {
37             SendAck(Rn);
38             AckNeeded = false;
39             NakSent = false;
40         }
41     }
42 }
43 }
44 }
```

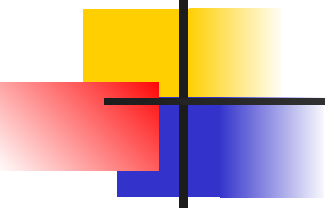
Piggybacking (Bidirectional transmission)

- *Is a method to combine a data frame with an acknowledgment.*
- *It can save bandwidth because data frame and an ACK frame can be combined into just one frame*

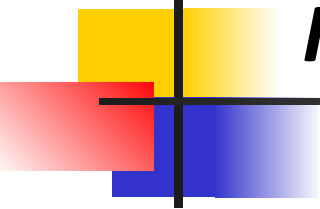


Communication between the host A and host B, three conditions can arise:

- When the host has **both data and the acknowledgment** to send, then it will attach the data along with the acknowledgment. In the above diagram, the host B will attach the data frame along with the acknowledgment of the last frame received from host A.
- When the host **does not have any data to send** then it will send only the acknowledgment. In the above diagram, when host A does not have any data frame to send. So, it will only send the acknowledgment of the last frame received.
- When the host has **only data to send** then it will send the data along with the acknowledgment of the last frame received. The duplicate acknowledgment will be discarded by the receiver and the data would be accepted.



HDLC



Introduction to HDLC

- The High-Level Data Link Control (HDLC) is part of the data link layer protocol in the OSI Model.
- The HDLC protocol embeds information in a data frame that allows devices to control data flow and correct errors
- HDLC is an ISO Standard developed from the Synchronous Data Link Control (SDLC) standard proposed by IBM
- Role of HDLC is to ensure that the data has been received without any loss or errors and in the correct order
- It is a bit-oriented protocol

Introduction to HDLC

- HDLC is applied for point-to-point and multipoint link structures based on the **bit-oriented** data format.



- It also applies the **Automatic Repeat Request (ARQ)** for performing **full-duplex communication** between network channels due to its high flexibility and reliable network structure.



HDLC Stations

To efficiently integrate and use the HDLC protocols, different stations are to be effectively applied in the channel:

Primary Station - It handles establishing and de-establishing the primary data channel to share frames in the network, known as commands. It also performs data management for the network



HDLC Stations

To efficiently integrate and use the HDLC protocols, different stations are to be effectively applied in the channel:

Secondary Station - They work under the command of the primary station, and the frames transmitted by this station are known as responses.



HDLC Stations

To efficiently integrate and use the HDLC protocols, different stations are to be effectively applied in the channel:

Combined Station - This network station can work as both the primary and secondary stations and handle commands and responses.



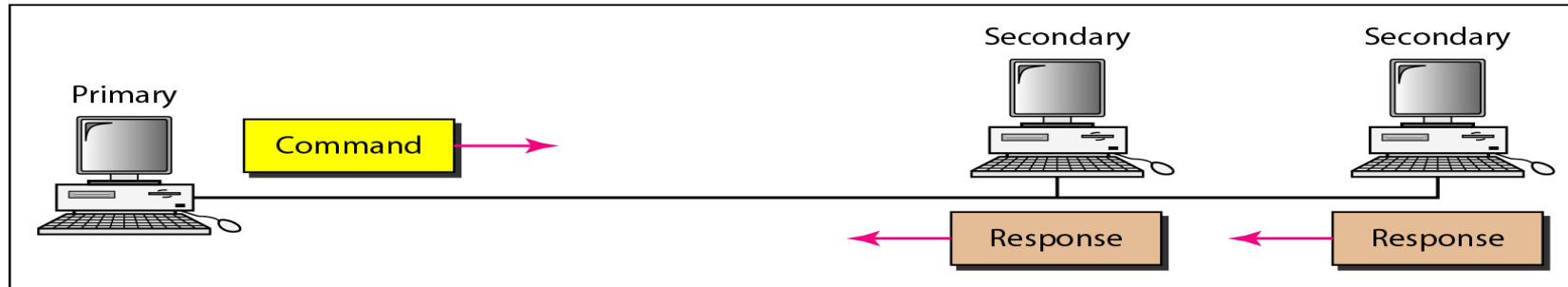
HDLC Transfer Models

The HDLC primarily supports two types of transfer models, which are:

Normal Response Model (NRM) - This transfer model combines primary and secondary stations in point-to-point or multipoint network configurations to exchange commands from primary stations and responses from secondary stations.



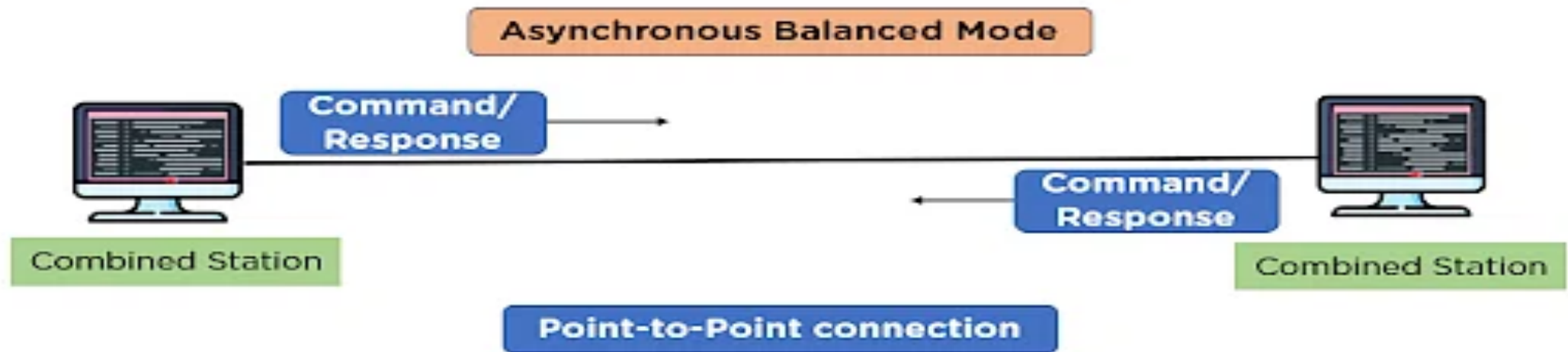
a. Point-to-point

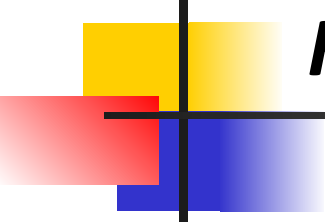


b. Multipoint

HDLC Transfer Models

Asynchronous Balanced Model (ABM) - In this transfer model, combined stations are installed in a point-to-point configuration for exchange commands and responses in a balanced format.





HDLC Transfer Models

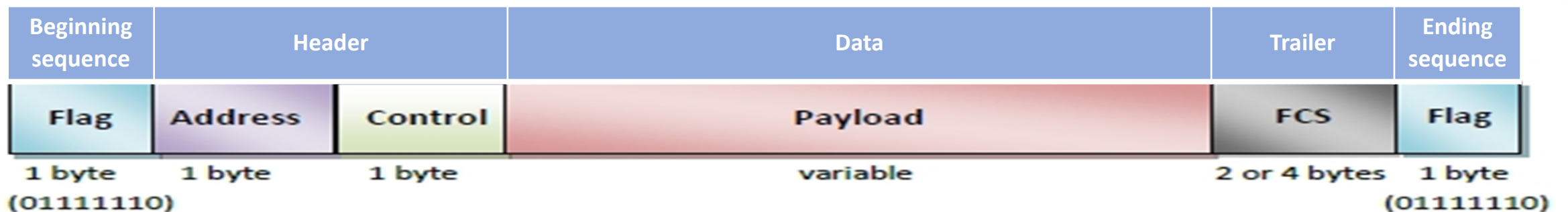
Asynchronous Response Model (ARM)

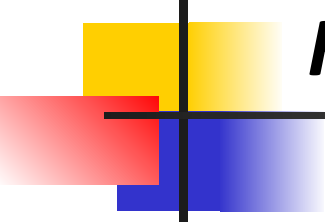
- Secondary station initiates a transmission without receiving permission from the primary station
- Primary terminal still retains responsibility for line initialization, error recovery, and logical disconnect
- Allows the secondary station to send frames asynchronously to the primary station

HDLC Frames and Types

The data unit for sharing information in HDLC is known as **frames**. HDLC frame consists of multiple frame fields, which may vary according to the frame type.

- **Flag Field** - In HDLC, each frame starts and ends with the flag field in the configuration and is defined by an **8-bit octet sequence 01111110** in the flag field.
- **Address Field** - It **encapsulates the receiver's address** in the field. For example: If the frame is sent from the primary station, it contains the secondary station's address and vice versa.
- **Control Field** - This field contains the **flow and error control information** in byte format.
- **Payload/Information Field** - It carries information from the network layer.
- **FCS Field** - This field stands for **Frame Check Sequence** and acts as an **error detection field** in the HDLC protocol, which includes a 16-bit CRC check bit.





HDLC Frames and Types

HDLC Frame Types:

- ❖ I – frame: Information frame
- ❖ S-frame: Supervisory frame
- ❖ U-frame: Un-numbered frame

Types of frame is determined by the control field.

I – frame	1st bit is 0	User Information
S-frame	1 st two bits is 10	Error and flow control
U-frame	1 st two bits is 11	Link management

HDLC Frames and Types

HDLC frames can be of the following three types, depending on the control field value of the frame.

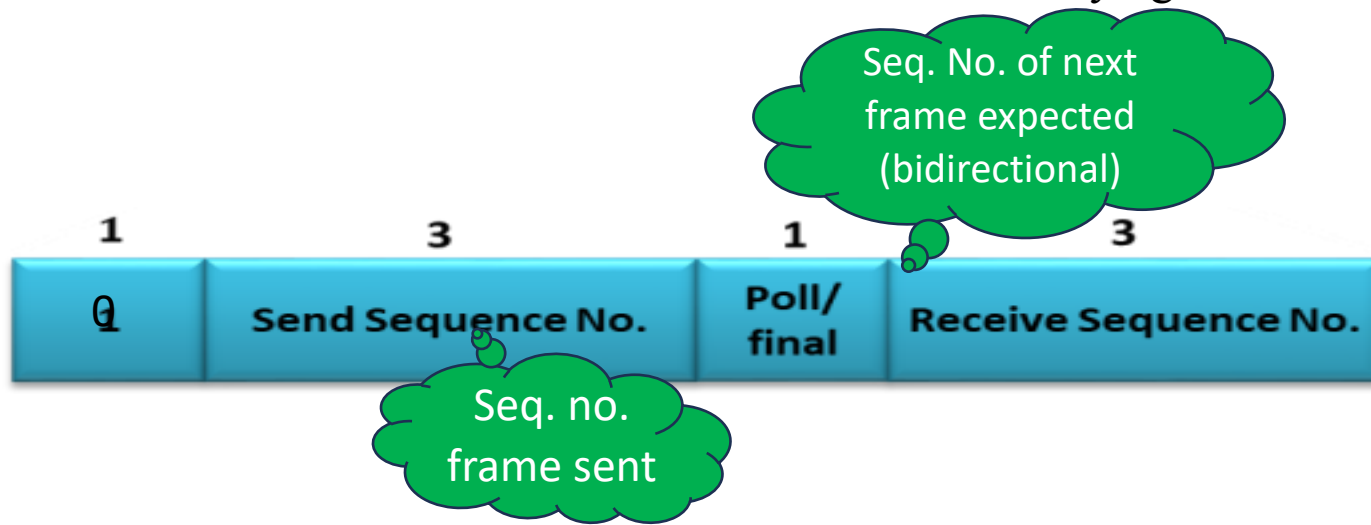
1. I-Frame

The I-Frame stands for **Information frame** and is applied to **encapsulate the user information** from the upper layer in the model and then transmit it into the network channel.



I-Frame Format

Note: The first bit of control field is 0 and is used for identifying the I-frame



- P/F field is a single bit with a dual purpose
- When it is set (bit = 1) mean Poll or Final
- Poll → means when the frame is sent by a primary station to secondary
- The address field contains the address of the receiver
- Final → means when the frame is send by a secondary to a primary
- When the address field contains the address of the sender

HDLC Frames and Types

2. S-Frame

The S-Frame stands for **Supervisory frame** and is used for error and data flow control, and **does not contain the information field** in the frame format.



S-Frame Format

Note: The two bits of control field is 10 and is used for identifying the S-frame

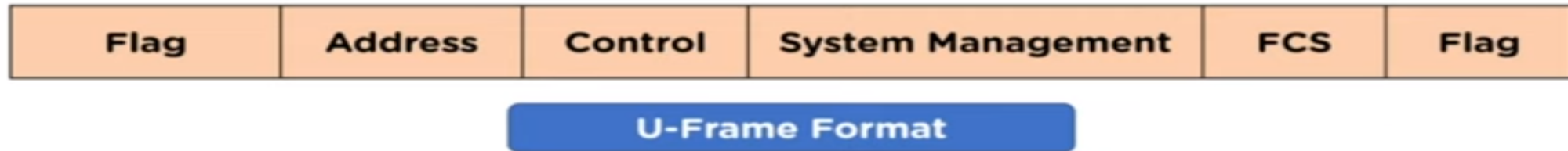


- S = 00 RR - Receiver Ready to accept more I-frames (data)
- S = 10 RNR - Receiver Not Ready to accept more I-frames
- S = 01 REJ - Go-Back-N retransmission request for an I-frame
- S = 11 SREJ - Selective retransmission request for an I-frame

HDLC Frames and Types

3. U-Frame

The U-Frame stands for **Un-numbered frame** used for **system management** and exchanging information between the connected network devices,



Note: The two bits of control field is 11 and is used for identifying the U-frame



- SNRM: set normal response mode (M1 = 00, M2 = 001)
- SABM: set asynchronous balanced mode (M1 = 11, M2 = 100)
- SABME: set asynchronous balanced mode, extended (M1 = 11, M2 = 110)
- DISC: disconnect (M1=00, M2=010)
- UA: un-numbered acknowledgement (M1 = 00, M2 = 110)
- RSET: resets send and receive sequence numbers (M1 = 11, M2=001)
- FRMR: frame reject (M1 = 10, M2=001)